Instituto Tecnológico y de Estudios Superiores de Occidente

Reconocimiento de validez oficial de estudios de nivel superior según acuerdo secretarial 15018, publicado en el Diario Oficial de la Federación del 29 de noviembre de 1976.

Department of Mathematics and Physics
Master of Data Science

Thesis Title

THESIS to obtain the **DEGREE** of
MASTER OF DATA SCIENCE

A thesis presented by:
Joey Ramone

Thesis Advisors:
Dr. Willie E. Coyote
Dr. Homer J. Simpson

Tlaquepaque, Jalisco, January, 2021

Instituto Tecnológico y de Estudios Superiores de Occidente

Reconocimiento de validez oficial de estudios de nivel superior según acuerdo secretarial 15018, publicado en el Diario Oficial de la Federación del 29 de noviembre de 1976.

Department of Mathematics and Physics

Master of Data Science Approval Form

Thesis Title: **Thesis Title**

Author: **Joey Ramone**

Thesis Approved to complete all degree requirements for the Master of Science Degree in Data Science.

Thesis Advisor, **Dr. Willie E. Coyote**

Thesis Co-Advisor, **Dr. Homer J. Simpson**

Thesis Reader, **Dr. Frederick Flintstone**

Thesis Reader, **Dr. Farrokh Bulsara**

Academic Advisor, **Dr. Maryam Mirzajani**

Tlaquepaque, Jalisco, January, 2021

Thesis Title

Joey Ramone

Abstract

Here

Thesis Title

Joey Ramone

Resumen

En esta página debes escribir el abstract pero en español.

Contents

Page

1 Introduction 19
- 1.1 Learning from Graph-Structured Data 19
- 1.2 Objectives and Scope 20
- 1.3 Structure of the Document 22

2 Learning Tasks on Graphs 23
- 2.1 Node-Level Tasks 24
- 2.2 Edge-Level Tasks 25
- 2.3 Graph-Level Tasks 26

3 Fundamentals of Graph Theory 27
- 3.1 Formal Definitions 27
- 3.2 The Adjacency Matrix and Graph Types 28
- 3.3 Walks, Connectivity, Homophily, and Isomorphism . 31
- 3.4 The Graph Laplacian and the Spectral Perspective . . 32
- 3.5 Notation Summary 34

4 Neural Networks and Their Limitations on Relational Data . . . 35
- 4.1 The Multilayer Perceptron 35
- 4.2 Training by Gradient Descent 36
- 4.3 Why the Multilayer Perceptron Fails on Graphs . . . 38

5 Inductive Bias, Invariance, and Equivariance. 41
- 5.1 Inductive Bias in Neural Architectures 41
- 5.2 The Relational Inductive Bias 42
- 5.3 Invariance and Equivariance 43
- 5.4 Permutation Invariance and Equivariance on Graphs 44

6 The Message-Passing Framework 47
- 6.1 From Relational Bias to Neighborhood Aggregation . 47
- 6.2 The Message-Passing Layer 49
  - 6.2.1 The Message Function 49
  - 6.2.2 The Aggregation Function 50
  - 6.2.3 The Update Function 50
  - 6.2.4 The General Equation of a Message Passing Layer 51
- 6.3 Stacking Layers: Receptive Field and Depth 52
- 6.4 Transductive and Inductive Learning 53
- 6.5 Expressive Power and the 1-WL Bound 54

6.6 The Design Space 55
6.6.1 Flexibility of the Framework 56
6.6.2 The MLP as a Degenerate Case 56
6.6.3 Established Architectures as Instances 57
7 Representative Architectures for Node-, Edge-, and Graph-Level Tasks 59
7.1 Node-Level Learning 59
7.1.1 From Embeddings to Predictions 60
7.1.2 The Graph Convolutional Network 60
7.2 Edge-Level Learning 62
7.2.1 The Encoder–Decoder Framework 63
7.2.2 The Graph Autoencoder 64
7.3 Graph-Level Learning 65
7.3.1 Pooling and Readout 65
7.3.2 The Graph Isomorphism Network 66
Bibliography 69

List of Figures

Page

1.1 Examples of graph-structured data. 20

2.1 The three levels of graph learning tasks. 23
2.2 Node-level tasks. 24
2.3 Edge-level tasks. 25
2.4 Graph-level tasks. 26

3.1 A graph and its adjacency matrices under two node indexings. 29
3.2 A directed graph and a weighted graph. 30
3.3 Self-loops, a multigraph, and a bipartite graph. 30
3.4 A homogeneous and a heterogeneous graph. 31
3.5 A walk and a shortest path. 31
3.6 A disconnected graph and its components. 32
3.7 Two isomorphic graphs. 32

4.1 The multilayer perceptron. 36
4.2 Neighborhoods of varying size. 38
4.3 The same graph under two node orderings. 39

5.1 Permutation invariance and equivariance on a graph. . . 45

6.1 The three functions of a message-passing layer. 52
6.2 The receptive field of a node grows with depth. 53

7.1 The encoder–decoder framework for link prediction. . . 64

List of Tables

Page

3.1 Notation introduced in this chapter. 34

Dedicated to:

Dedicado a:

Escribir la dedicatoria pero en español.

1 Introduction

Contents

1.1 Learning from Graph-Structured Data 19
1.2 Objectives and Scope 20
1.3 Structure of the Document 22

1.1 Learning from Graph-Structured Data

Data in machine learning often comes in a few familiar forms. Most datasets are tabular: a table of rows and columns in which each row is an independent example described by the same fixed set of features. Other data is organized as images, that is, grids of pixels, or as sequences such as text and audio, whose elements follow one another in a specific order. What these forms share is regularity: each has a fixed, well-defined structure that repeats across every example.

Many real-world systems do not fit any of these shapes. A molecule is a set of atoms connected by chemical bonds, and neither the number of atoms nor the way they are connected is the same from one molecule to the next. A social network is a set of people connected by relationships, where each person may be linked to a different number of others. A citation network is a set of papers that reference one another, each citing a different set of others. These are not tables, images, or sequences; they are networks of entities and the relations between them (Figure 1.1). Data of this kind is naturally described by a *graph*: a set of entities, called *nodes*, together with a set of connections between them, called *edges*. Both nodes and edges can carry their own data, for example in a molecule, each node is an atom labeled with its element (carbon, hydrogen, oxygen, and so on) and each edge is a bond of a particular type (single, double, and so on); in a social network, the nodes are people and the edges are the relationships among them.

Rather than treating a domain as a collection of isolated data points, we describe it in terms of entities and the relationships among them, which means there is an underlying graph whose structure ties those

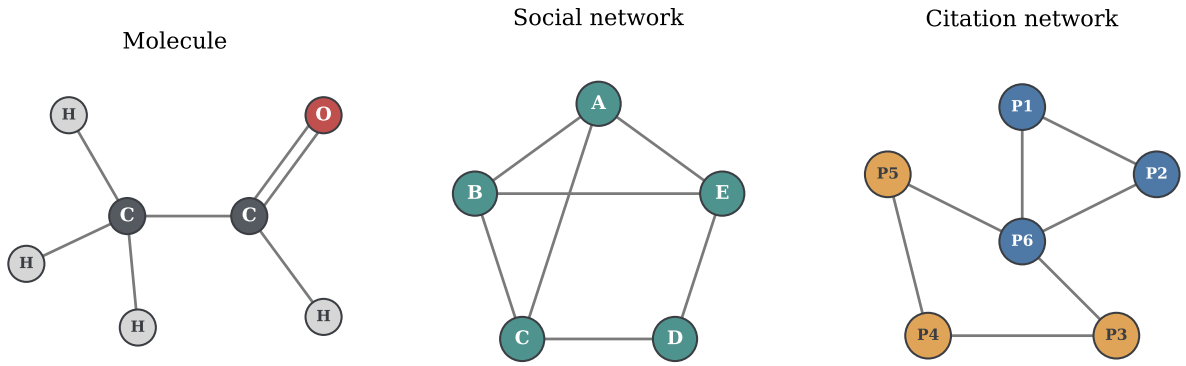

Figure 1.1: Three domains represented as graphs. A molecule, whose nodes are atoms labeled by their element and whose edges are bonds (single or double); a social network, whose nodes are people and whose edges are relationships; and a citation network, whose nodes are papers (colored by topic) and whose edges are citations between them. In each case the entities and their connections, not a table, grid, or sequence, define the data.

entities together. Seen this way, the graph is not merely one possible encoding of a molecule or a social network but a faithful representation: it records exactly which entities exist and how they relate, without forcing the data into a shape it does not have. A wide range of domains exhibit this kind of relational structure, and modeling them explicitly as graphs, rather than ignoring the connections, can lead to more accurate predictions. Data organized in this way is called *graph-structured data*.

Most deep learning models are tailored to the regular structures described above, particularly sequences and images laid out on a grid. Graphs are harder to work with. They have no fixed size, since different graphs contain different numbers of nodes. They have no fixed layout: a pixel in a grid has a clear sense of place, with neighbors above, below, and to either side, whereas a node in a graph has no such spatial arrangement and may be connected to any number of others. And they have no natural ordering, with no first or last node and no canonical point from which to read the rest.

The goal, then, is a model that takes a graph as input and produces predictions about it, such as a label for each node, wheter connections exist between pairs of nodes, or a single label for the graph as a whole. This raises the central question: how can a predictive model – concretely, a neural network – be designed to operate directly on a graph, respecting its variable size and its lack of order instead of discarding them? Neural networks built for this purpose are called *Graph Neural Networks*, hereafter abbreviated as GNNs.

1.2 *Objectives and Scope*

The general objective of this work is to provide a self-contained reference on GNNs for homogeneous graphs under the *message-passing* framework: a single document that develops the theory that motivates

and defines these models, presents representative architectures, and evaluates them through reproducible implementations.

Specifically, the work aims to:

- establish the foundations that motivate GNNs: how graphs represent relational data, why standard neural networks cannot learn from it directly, and the inductive biases a model on graphs must respect;
- develop the message-passing framework as the unifying formulation from which GNN architectures are derived;
- present representative architectures for the three standard learning tasks on graphs: the Graph Convolutional Network (GCN) for node classification, the Graph Auto-Encoder (GAE) for link prediction, and the Graph Isomorphism Network (GIN) for graph classification;
- implement the three architectures on standard benchmark datasets (Cora, Amazon Computers, and NCI1) using PyTorch Geometric, with reproducible implementations, and evaluate each against a multilayer perceptron baseline that ignores the graph, examining the predictive benefit of explicitly modeling the relational structure of the data;
- examine the principal limitations of GNNs under this framework, including *oversmoothing* and the ceiling on their expressive power.

Its contribution is the reference document itself: a single, reproducible source that develops the message-passing framework from first principles, presents representative GNN architectures, and serves as a foundational resource for studying the field.

The graphs we consider are treated mainly as *undirected*, meaning their connections carry no direction, and *simple*, meaning that at most one connection joins any pair of nodes and that no node is connected to itself. This choice reflects simplicity rather than a limitation of the framework: message passing has enough design space to accommodate *directed* graphs, *self-loops* (a node connected to itself), and even *multigraphs* (more than one connection between the same pair of nodes).

The graphs considered in this work are also *homogeneous*: they contain a single kind of node and a single kind of connection. This restriction is deliberate. The homogeneous setting is where the message-passing framework is most directly understood, and it is the foundation on which more elaborate models are built, including *heterogeneous* graphs, which carry several kinds of nodes and connections and require machinery beyond the scope of this work. Establishing the homogeneous case is a prerequisite for these extensions, which we

leave as future research. The formal definitions of these graph types are given later in the document.

1.3 *Structure of the Document*

This introduction has set out the motivation, objectives, and scope of the work. The remainder of the document is organized as follows.

The first chapters establish the foundations. Chapter 2 surveys the tasks that graph learning addresses at the node, edge, and graph level, together with the practical applications that motivate the rest of the work. Chapter 3 establishes the formal definitions and notation for graphs used throughout. Chapter 4 reviews the multilayer perceptron and shows why standard neural networks are ill-suited to relational data. Chapter 5 develops the notion of inductive bias and the permutation invariance and equivariance that any graph layer must satisfy.

Chapter 6 builds on these foundations to derive the message-passing framework: the message, aggregation, and update functions, the receptive field that successive layers induce, the expressive power of the resulting models, and the architectural design space it enables.

Chapter 7 presents the three representative architectures within this framework: GCN for node classification, GAE for link prediction, and GIN for graph classification. Chapter **??** turns to practice, providing reproducible implementations of these architectures on the Cora, Amazon Computers, and NCI1 benchmarks in PyTorch Geometric, with the accompanying code available in a companion repository.

Chapter **??** examines the limitations of GNNs under the message-passing framework, and Chapter **??** synthesizes the work and outlines directions for future research.

2 Learning Tasks on Graphs

Contents

2.1 Node-Level Tasks 24
2.2 Edge-Level Tasks 25
2.3 Graph-Level Tasks 26

Once a domain has been represented as graph-structured data, a natural question arises: what machine learning tasks can be defined on a graph? A graph can be viewed at three levels of granularity: its individual nodes, its edges, and the graph as a whole. Each level gives rise to a family of prediction tasks: predicting a property of each node, predicting the existence or type of a connection between pairs of nodes, or predicting a property of the entire graph. These are known, respectively, as *node-level*, *edge-level*, and *graph-level tasks* (Figure 2.1), and they structure the architectures and implementations developed in the rest of this work.

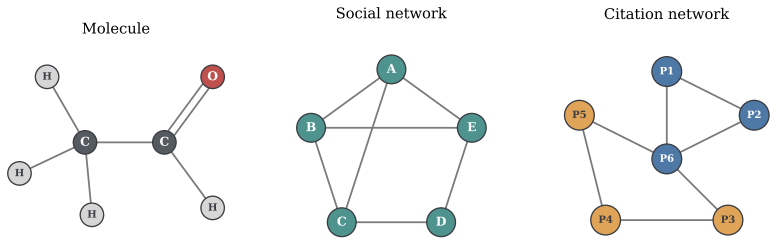

Figure 2.1: The three levels at which a prediction can be made on a single graph. A node-level task predicts a property of an individual node (highlighted); an edge-level task predicts a property of a connection between two nodes, such as whether it exists (dashed); and a graph-level task predicts a single property of the entire graph (framed). The same object supports all three.

These three levels correspond to the standard categories of graph learning tasks, but they do not exhaust the field. Other settings include *community/subgraph-level tasks*, which focus on groups of related nodes, and *graph generative tasks*, which aim to generate new graphs, such as candidate molecular structures with desirable properties. Although these settings rely on the same representational principles introduced here, they lie outside the scope of this work, which focuses on the three standard task levels.

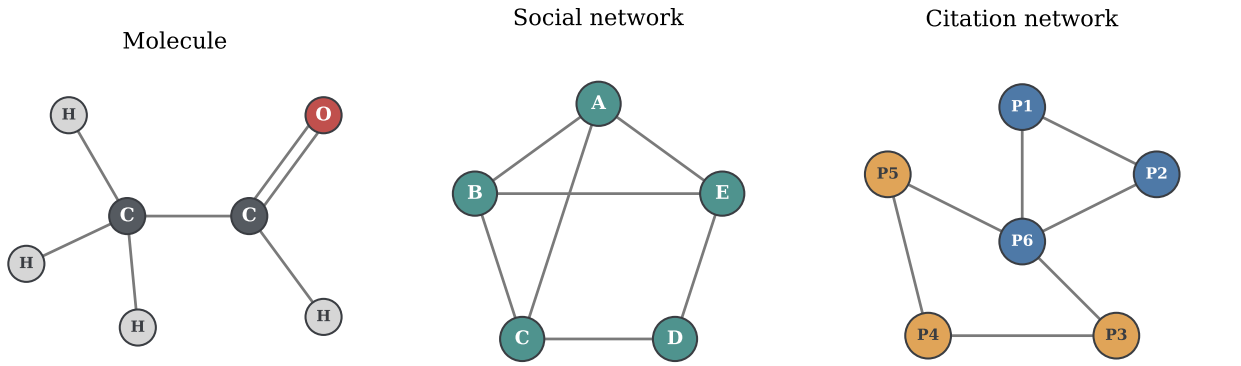

Figure 2.2: Left: the node classification task in the abstract. Some nodes carry a known label (shown by color) while others are unknown (marked with a question mark); the model predicts a label for every node from its attributes and the surrounding structure. Right: a concrete instance as a citation network, where each node is a scientific paper and the predicted label is its topic.

The following sections examine each of these levels in turn and present representative applications that motivate them.

2.1 Node-Level Tasks

A *node-level task* asks for a prediction about each individual node of a graph, drawing on both the node's own attributes and its position within the surrounding structure. The most common instance is *node classification*: assigning each node to one of a set of categories. A representative setting is a citation network, in which the nodes are scientific papers, the edges are citations between them, and the goal is to infer the topic of each paper (Figure 2.2). A paper's own text is informative, but so is its neighborhood: papers tend to cite others on related subjects, so the topics of a paper's neighbors are strong evidence for its own. Node classification exploits exactly this, combining what a node contains with what surrounds it. The same pattern appears whenever entities in a network must be categorized.

The target need not be a category. A graph representation of a protein, for example, can be constructed where nodes represent amino acid residues and edges connect residues that are close in space. One may predict a discrete property of each residue, such as its functional role, or a continuous property, such as its position in three dimensions, a formulation that underlies recent advances in protein structure prediction [1]. A node-level task can therefore be classification or regression, depending on whether the per-node target is categorical or numerical.

The same framing extends to economic settings. In a financial network whose nodes are accounts and whose edges are transactions between them, fraud detection amounts to flagging each account as legitimate or fraudulent: a node classification problem in which

[1] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873): 583–589, 2021. DOI: 10.1038/s41586-021-03819-2

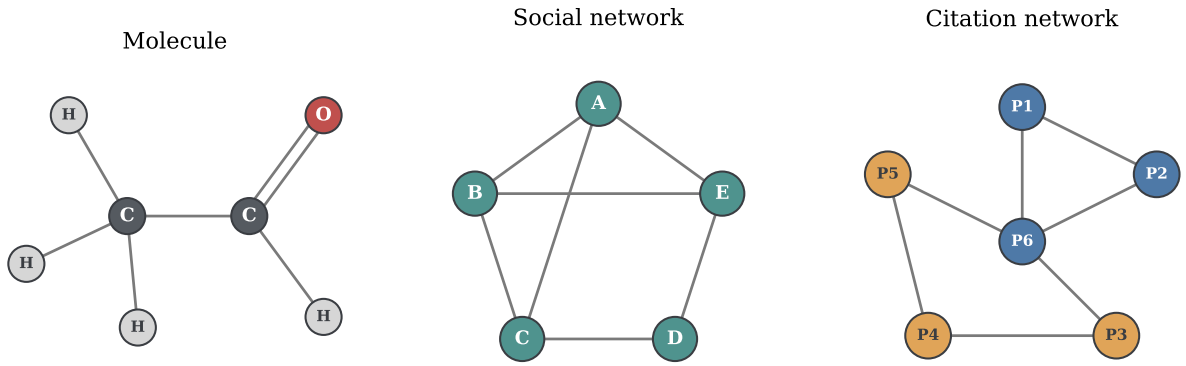

Figure 2.3: Left: the link prediction task in the abstract. Solid edges are observed connections; dashed edges marked with a question mark are candidate connections whose existence the model must predict. Right: a concrete instance as a recommendation setting, where the two kinds of entity (users, drawn as squares, and products, drawn as circles) are joined by observed interactions, and the task is to predict further user–product links.

fraudulent behavior is often more evident in the surrounding network of connections than in any single account considered in isolation.

2.2 *Edge-Level Tasks*

An *edge-level task* focuses on the relationships between entities rather than the entities themselves. The central example is *link prediction*: given the edges already present in a graph, predict which further edges are likely to exist. Recommendation is one of the the most familiar application (Figure 2.3). In its simplest form, a network records products that are frequently bought together, with the nodes being products and edges indicating co-purchases. Predicting missing edges in this network amounts to identifying product pairs that are likely to be associated, even if they have not yet been observed together. A richer version introduces two kinds of entity, users and products, with an edge representing interactions such as purchases or ratings; here, link prediction becomes the recommendation problem itself, anticipating which products a user will want next.

What makes the graph formulation effective is that it learns from the relational structure rather than treating each entity in isolation. A recommender system that treats products only by their individual attributes, ignoring the network of interactions around them, discards precisely the signal that reveals which items behave similarly; methods that propagate information along the graph structure consistently outperform such approaches, a principle that underlies graph-based recommender systems deployed at web scale [2].

Link prediction extends well beyond commerce. Given a pair of drugs, one may predict whether taking them together produces an adverse interaction, by placing both within a larger biological network and asking whether a particular kind of connection should join them [3].

[2] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 974–983, 2018. DOI: 10.1145/3219819.3219890

[3] Marinka Zitnik, Monica Agrawal, and Jure Leskovec. Modeling polypharmacy side effects with graph convolutional networks. *Bioinformatics*, 34(13):i457–i466, 2018. DOI: 10.1093/bioinformatics/bty294

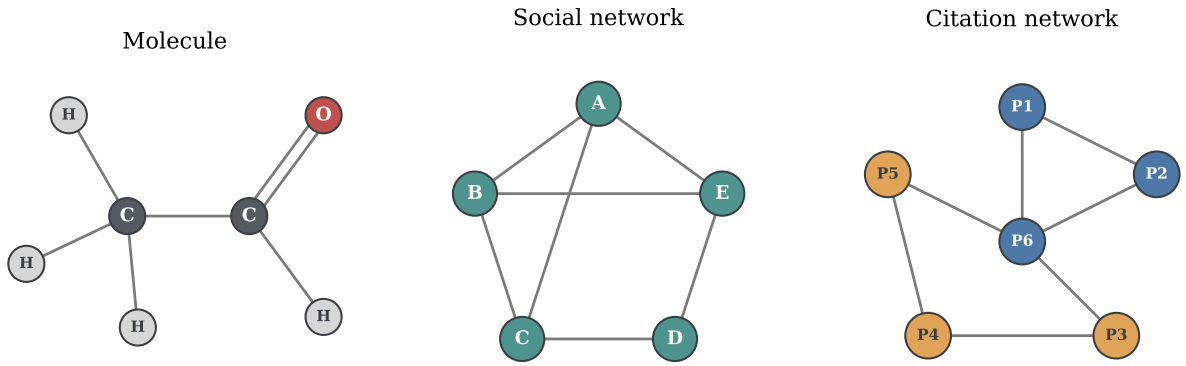

Figure 2.4: Left: the graph classification task in the abstract. The data is a collection of distinct graphs, differing in size and shape, each assigned a single label for the graph as a whole. Right: a concrete instance as molecular property prediction, where each graph is a molecule (atoms as nodes, bonds as edges) labeled by a property such as being active or inactive against a target.

The same formulation is also used to complete *knowledge graphs*, large networks of facts in which entities are connected by relations: many true relations are simply unobserved, and link prediction proposes the most plausible ones.

2.3 Graph-Level Tasks

A *graph-level task* treats an entire graph as a single example and predicts a property of the whole. The representative task is *graph classification*: assigning a label to a complete graph (Figure 2.4). Here the data is not one large network but a collection of many separate graphs, each with its own label, and the model must summarize the structure and attributes of an entire graph into a single prediction. Molecular property prediction is the canonical example. A molecule is naturally a graph, with atoms as nodes and chemical bonds as edges, and many prediction tasks about it are graph-level: whether a compound is toxic, or whether it is active against a given biological target. Because molecules vary in size and structure, this is exactly the regime that calls for a model able to handle graphs of varying structure and return one prediction per graph.

This capability feeds directly into drug discovery, where large libraries of candidate compounds are screened to prioritize the few worth synthesizing and testing in the laboratory, a process in which an accurate graph-level predictor can save considerable experimental effort. The same idea applies in materials science, where the atomic structure of a material is represented as a graph and a graph-level model predicts a physical property such as its stability or its mechanical strength.

Across all three levels, the graph is used in different ways, with the level of the task determining what is predicted and how a model must process the structure to predict it.

3 Fundamentals of Graph Theory

Contents

3.1 Formal Definitions 27
3.2 The Adjacency Matrix and Graph Types 28
3.3 Walks, Connectivity, Homophily, and Isomorphism 31
3.4 The Graph Laplacian and the Spectral Perspective 32
3.5 Notation Summary 34

The applications surveyed in Chapter 2 share a single underlying object: the graph. To move from describing tasks to formulating them precisely, that object must itself be defined formally. This chapter introduces the formal language of graphs used throughout the rest of this work. It first defines graphs, neighborhoods, and node and edge features; it then encodes graph structure algebraically through the adjacency matrix and distinguishes the main types of graphs; it examines the structural properties, from walks to isomorphism, on which later developments rely; it introduces the graph Laplacian and the spectral perspective it provides; and it closes with a summary of the notation fixed here.

3.1 Formal Definitions

A *graph* is a pair

$$\mathcal{G} = (V, E), \tag{3.1}$$

where V is a finite set of *nodes*, also called vertices, and E is a set of *edges*, also called links, each one an unordered pair of nodes recording which two nodes are connected [1]. The number of nodes is denoted $n = |V|$ and the number of edges $m = |E|$. In classical graph-theoretic notation, generic nodes are denoted by $u, v \in V$, and an edge between them by the unordered pair $\{u, v\} \in E$; at this stage, the notation simply states that the two nodes are connected.

Because V and E are sets in the mathematical sense, neither carries an intrinsic order: a graph specifies which nodes are connected, and

[1] Reinhard Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, 5 edition, 2017. ISBN 978-3-662-53622-3

nothing more. It is nevertheless convenient, particularly when graphs are represented by matrices, to fix an indexing of the nodes by integers. Throughout this work, nodes are therefore denoted $i, j \in \{1, \dots, n\}$, with the convention that i refers to the node under consideration and j to a neighboring node whenever a neighborhood relation is involved. The choice of indexing is arbitrary and carries no information about the graph itself, a point this chapter returns to.

Two nodes are *adjacent*, or *neighbors*, when an edge connects them. The *neighborhood* of a node i is the set of its neighbors,

$$\mathcal{N}(i) = \{\, j \in V : \{i, j\} \in E \,\}, \tag{3.2}$$

and the *degree* of node i is the number of neighbors it has,

$$\deg(i) = |\mathcal{N}(i)|. \tag{3.3}$$

In a social network, for example, the neighborhood of a person is the set of people connected to them, and the degree counts them.

Beyond its structure, a graph typically carries data [2]. Each node i has an associated *node feature vector* $\mathbf{x}_i \in \mathbb{R}^d$, where d is the node feature dimension, describing the attributes of the entity the node represents, such as the textual content of a paper in a citation network, or the chemical element of an atom in a molecule. Stacking the n node feature vectors as rows produces the *node feature matrix*

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d}, \tag{3.4}$$

whose i-th row is the transposed feature vector of node i. Writing the features as a matrix presupposes that the node indexing is fixed as above. Edges can carry attributes as well: the *edge feature vector* $\mathbf{e}_{i,j} \in \mathbb{R}^c$, where c is the edge feature dimension, describes the connection between nodes i and j, such as the type of a chemical bond in a molecule.

[2] William L. Hamilton. *Graph Representation Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool, 2020. ISBN 978-1-68173-965-6

3.2 The Adjacency Matrix and Graph Types

With the nodes indexed from 1 to n, the structure of a graph can be encoded in a single matrix. The *adjacency matrix* of $\mathcal{G}$ is the matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ with entries

$$A_{ij} = \begin{cases} 1 & \text{if } \{i, j\} \in E, \\ 0 & \text{otherwise,} \end{cases} \tag{3.5}$$

so that each entry records the presence or absence of a single edge.

The adjacency matrix depends on the node indexing. Reordering

There are $n!$ possible indexings of n nodes, and each yields a valid adjacency matrix of the same graph.

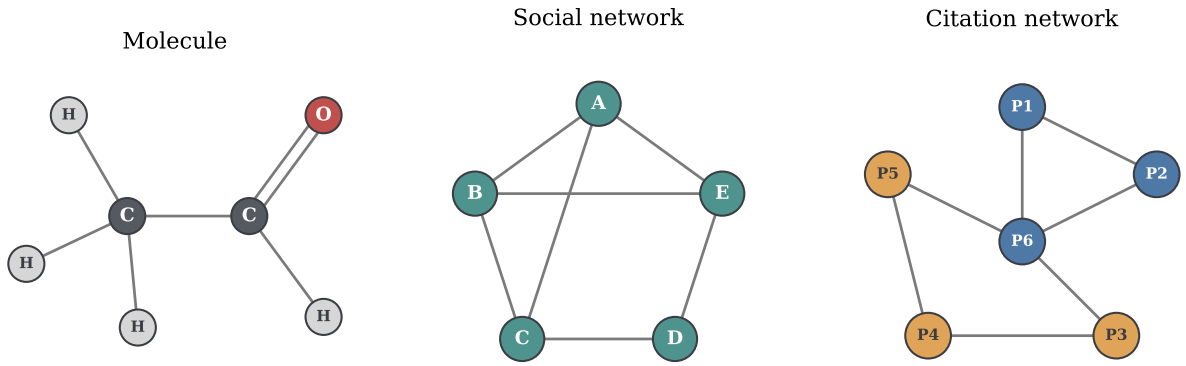

Figure 3.1: A graph on six nodes (left) and its adjacency matrices under two different indexings of the nodes (center and right). Both matrices describe the same structure: reordering the nodes permutes the rows and columns of $\mathbf{A}$.

the nodes permutes the rows and columns of $\mathbf{A}$, producing a different matrix that describes exactly the same graph. Figure 3.1 shows a graph together with its adjacency matrices under two different indexings: a graph has not one adjacency matrix but one per indexing, and no single one of them is privileged.

The first distinction among graph types concerns the orientation of edges. In an *undirected* graph, the setting assumed so far, an edge $\{i, j\}$ has no orientation and can be traversed in either direction; its adjacency matrix is consequently symmetric, $\mathbf{A} = \mathbf{A}^\top$. In a *directed* graph, edges are instead ordered pairs: the pair $(i, j) \in E$ denotes an edge from a *source* node i to a *target* node j, and the presence of (i, j) no longer implies the presence of (j, i). The adjacency entry reads $A_{ij} = 1$ exactly when $(i, j) \in E$, so $\mathbf{A}$ is in general not symmetric and row i lists the targets of the edges leaving node i. The degree of a node accordingly splits in two: its *out-degree*, the number of edges leaving it, and its *in-degree*, the number of edges arriving at it. Unless stated otherwise, the graphs treated in this work are undirected.

A second distinction concerns whether edges carry a magnitude. In a *weighted* graph, each edge has an associated scalar weight $w_{ij} \in \mathbb{R}$, expressing, for instance, the strength of a relationship or a physical distance; the adjacency matrix then stores $A_{ij} = w_{ij}$ in place of 1 (Figure 3.2). In an *unweighted* graph, every edge counts the same and (3.5) applies as stated. A scalar weight can be viewed as the special case $c = 1$ of the edge feature vectors introduced in Section 3.1, that is, a single number attached to each edge.

A *self-loop* is an edge connecting a node to itself; in the adjacency matrix it appears as a nonzero diagonal entry, $A_{ii} \neq 0$. A *multigraph* allows several parallel edges between the same pair of nodes, a multiplicity that the binary entries of (3.5) cannot record. A graph with no self-loops and at most one edge between any pair of nodes

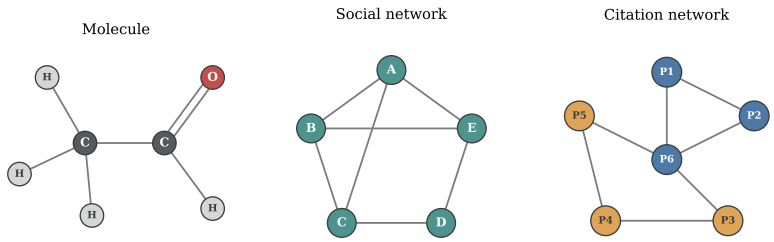

Figure 3.2: Left: a directed version of the graph of Figure 3.1 and its adjacency matrix. Edge orientations make $\mathbf{A}$ asymmetric: row i lists the targets of the edges leaving node i, and the in-degree and out-degree of one highlighted node are annotated. Right: a weighted graph, with edge thickness proportional to the weight w_{ij}.

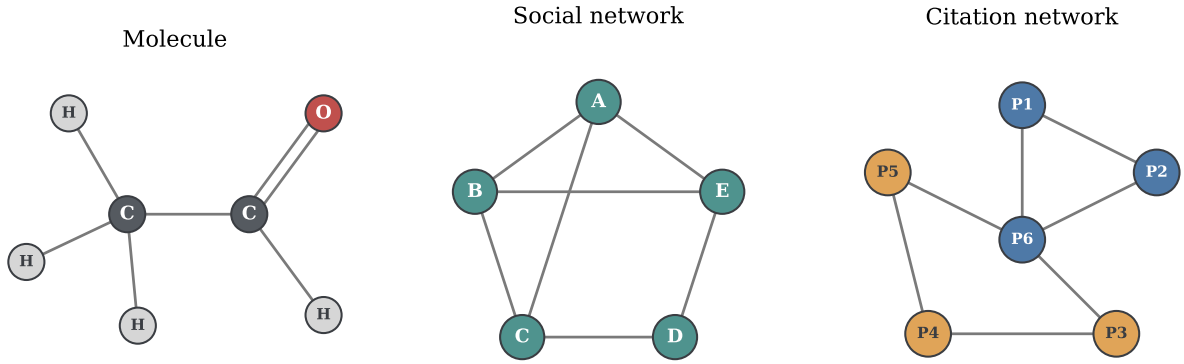

Figure 3.3: Left: a graph with self-loops. Center: a multigraph with parallel edges. Right: a bipartite graph, with its two subsets of nodes drawn as distinct shapes.

is called *simple*. A *bipartite* graph, in contrast, is not a departure from simplicity but a structural subclass of it: its nodes partition into two disjoint subsets, and every edge connects a node of one subset with a node of the other, as in a network of users and products joined by purchases. Because of this specificity, such graphs are explicitly named bipartite; a graph described only as simple implies no such partition. Figure 3.3 illustrates the three cases. As established in Chapter 1, the graphs treated in this work are simple.

The last distinction concerns the kinds of entities and relations a graph contains. A graph is *homogeneous* when all of its nodes represent one kind of entity and all of its edges represent one kind of relation: every node then shares the same feature space $\mathbb{R}^d$ and every edge the same feature space $\mathbb{R}^c$. A citation network is homogeneous, since every node is a paper and every edge a citation. A graph is *heterogeneous* when it contains several types of nodes or edges, each with its own semantics and, typically, its own feature space [3]. A citation network that includes both papers and their authors is heterogeneous: papers and authors are different kinds of entities, joined by a relation of authorship. Figure 3.4 contrasts the two settings. As stated in Chapter 1, this work treats homogeneous graphs.

The homogeneous/heterogeneous terminology comes from the network analysis and machine learning literature rather than from classical graph theory.

[3] William L. Hamilton. *Graph Representation Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool, 2020. ISBN 978-1-68173-965-6

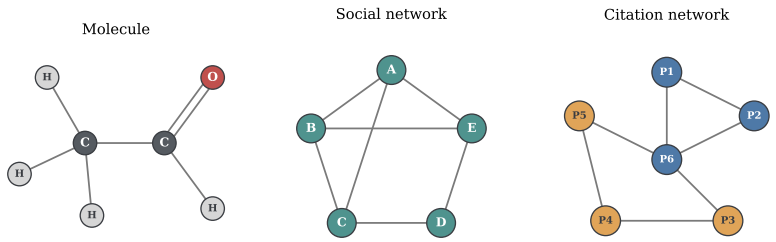

Figure 3.4: Left: a homogeneous graph, in which all nodes represent the same kind of entity and all edges the same relation. Right: a heterogeneous graph, in which node shapes denote different entity types and edge styles denote different relations.

3.3 Walks, Connectivity, Homophily, and Isomorphism

The definitions so far describe a graph locally: nodes, their neighbors, and their attributes. Several properties that matter in the remainder of this work are instead global, arising from how edges chain together across the graph. A *walk* is a sequence of nodes in which each consecutive pair is joined by an edge, and its *length* is the number of edges traversed; a walk may revisit nodes and edges. A *path* is a walk in which all nodes are distinct (Figure 3.5).

The *geodesic distance* between two nodes is the length of a shortest path between them,

$$d(i,j) = \min\Big\{k \in \mathbb{N} : \text{there is a path of length } k \text{ between } i \text{ and } j\Big\} \tag{3.6}$$

with $d(i,i) = 0$. The distance is always written with its two arguments, $d(i,j)$, which distinguishes it from the node feature dimension d. When no path joins i and j, the distance is taken to be infinite, a situation characterized next.

A graph is *connected* when a path exists between every pair of its nodes, that is, when $d(i,j)$ is finite for all $i, j \in V$. When it is not, the graph decomposes into *connected components*: maximal groups of nodes that are mutually reachable, with no edges between groups (Figure 3.6). In directed graphs, connectivity refines into weak and strong variants according to whether edge orientations are respected; since this work treats undirected graphs, the refinement is not developed further.

Structure and attributes are often correlated. A graph exhibits *homophily* when adjacent nodes tend to have similar attributes, and *heterophily* when they tend to differ. The notion originates in the analysis of social networks, where similar individuals form ties at higher rates than dissimilar ones [4], and it extends to many of the graphs of Chapter 2: in a citation network, a paper predominantly cites papers on related topics, so neighboring nodes tend to share a subject. Whether a graph is homophilous or heterophilous is an

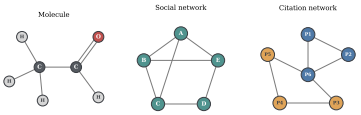

Figure 3.5: A walk that revisits a node (top) and a shortest path between the same endpoints (bottom), whose length is their geodesic distance.

[4] Miller McPherson, Lynn Smith-Lovin, and James M. Cook. Birds of a feather: Homophily in social networks. *Annual Review of Sociology*, 27(1):415–444, 2001. DOI: 10.1146/annurev.soc.27.1.415

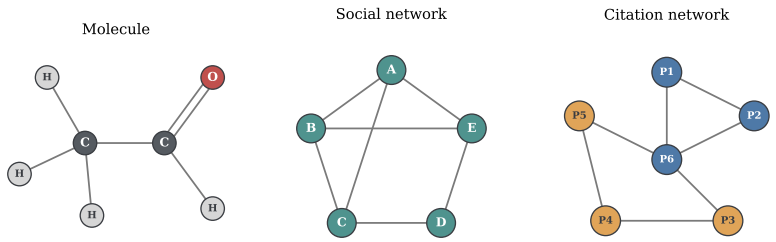

Figure 3.6: A disconnected graph with two connected components, one of which is a single isolated node.

empirical property of the data rather than a structural constraint, and the degree to which it holds varies from one graph to another.

The arbitrariness of node indexing observed in Section 3.2 raises a final question: when do two graphs that are presented differently have the same structure? Two graphs $\mathcal{G}_1 = (V_1, E_1)$ and $\mathcal{G}_2 = (V_2, E_2)$ are *isomorphic* when there exists a bijection $\pi : V_1 \to V_2$ that preserves adjacency,

$$\{i, j\} \in E_1 \quad \Longleftrightarrow \quad \{\pi(i), \pi(j)\} \in E_2, \tag{3.7}$$

for all nodes $i, j \in V_1$. Isomorphic graphs are the same graph up to a relabeling of its nodes; the two adjacency matrices of Figure 3.1 are representations related in exactly this way. Two graphs drawn very differently may be isomorphic (Figure 3.7). Determining whether two graphs are isomorphic is a fundamental problem in graph theory. A classical heuristic, the Weisfeiler–Leman test, distinguishes most non-isomorphic graphs in practice [5], and it reappears later in this work in connection with the expressive power of graph neural networks.

[5] Christopher Morris, Yaron Lipman, Haggai Maron, Bastian Rieck, Nils M. Kriege, Martin Grohe, Matthias Fey, and Karsten Borgwardt. Weisfeiler and Leman go machine learning: The story so far. *Journal of Machine Learning Research*, 24(333):1–59, 2023. http://jmlr.org/papers/v24/22-0240.html

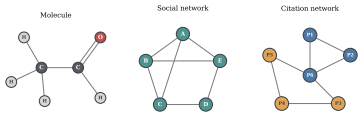

Figure 3.7: Two graphs that are drawn differently but are isomorphic: a bijection between their node sets preserves all adjacencies.

3.4 The Graph Laplacian and the Spectral Perspective

The structural properties of Section 3.3 and the algebraic encoding of Section 3.2 meet in a single observation: powers of the adjacency matrix count walks. For any positive integer k, the entry in row i and column j of the k-th matrix power gives the number of walks of length k between the two nodes,

$$(\mathbf{A}^k)_{ij} = \big|\{\,\text{walks of length } k \text{ between } i \text{ and } j\,\}\big|, \tag{3.8}$$

a count that is symmetric in i and j for the undirected graphs treated here. In particular, the entry is nonzero exactly when the two nodes are joined by some walk of that length, which ties the reachability notions of Section 3.3 directly to matrix algebra and motivates the matrices introduced next.

The *degree matrix* $\mathbf{D} \in \mathbb{R}^{n \times n}$ is the diagonal matrix that holds the node degrees,

$$\mathbf{D} = \operatorname{diag}\big(\deg(1), \ldots, \deg(n)\big), \tag{3.9}$$

with every off-diagonal entry equal to zero. Combining it with the adjacency matrix yields the central operator of spectral graph theory [6], the *graph Laplacian*

$$\mathbf{L} = \mathbf{D} - \mathbf{A}. \tag{3.10}$$

Each diagonal entry of $\mathbf{L}$ is the degree of the corresponding node, and each off-diagonal entry is -1 where an edge is present and 0 otherwise. For the six-node graph of Figure 3.1, under the node indexing of its centre panel and with $\mathbf{D} = \operatorname{diag}(2,3,2,3,3,1)$, the Laplacian is

$$\mathbf{L} = \begin{bmatrix} 2 & -1 & 0 & 0 & -1 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ -1 & -1 & 0 & -1 & 3 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{bmatrix}.$$

The Laplacian admits a direct interpretation in terms of how values spread over a graph. Assigning a real number to each node defines a vector $\mathbf{f} \in \mathbb{R}^n$ called a *graph signal*, with component f_i the value at node i. The Laplacian measures how much such an assignment varies across edges through the quadratic form

$$\mathbf{f}^\top \mathbf{L} \mathbf{f} = \sum_{\{i,j\} \in E} (f_i - f_j)^2, \tag{3.11}$$

which sums the squared difference between the values at the two endpoints of every edge. The form vanishes when $\mathbf{f}$ is constant on each connected component and grows as the values assigned to neighboring nodes diverge. Because it is a sum of squares, the quadratic form is nonnegative for every $\mathbf{f}$, so the Laplacian is *positive semidefinite*.

Being real and symmetric, the Laplacian admits an eigendecomposition

$$\mathbf{L} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top, \tag{3.12}$$

where the columns of $\mathbf{U} \in \mathbb{R}^{n \times n}$ form an orthonormal basis of eigenvectors and $\mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ collects the corresponding eigenvalues. Symmetry guarantees that these eigenvalues are real, and positive semidefiniteness that they are nonnegative; ordered, they satisfy $0 = \lambda_1 \leq \cdots \leq \lambda_n$. The smallest eigenvalue is always zero, with the constant vector as an eigenvector, since the rows of $\mathbf{L}$ sum to zero. More can be said: the multiplicity of the eigenvalue 0 equals the number of connected components of the graph, linking the spectral description back to the connectivity of Section 3.3.

For many purposes the Laplacian is rescaled by the node degrees. The *symmetric normalized Laplacian* is

$$\mathbf{L}_{\mathrm{sym}} = \mathbf{D}^{-1/2} \mathbf{L} \mathbf{D}^{-1/2} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}, \tag{3.13}$$

[6] Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, 1997. ISBN 978-0-8218-0315-8

where $\mathbf{I}$ is the identity matrix and $\mathbf{D}^{-1/2}$ is the diagonal matrix of inverse square-root degrees. It is again symmetric and positive semidefinite, so it shares the spectral structure just described, while its eigenvalues are confined to the interval $[0, 2]$ [7]. This normalized form is the variant used in the spectral developments of later chapters. Together, these properties make the Laplacian a natural bridge between the structure of a graph and the linear algebra used to operate on it.

For a node of degree zero the corresponding entry of $\mathbf{D}^{-1/2}$ is taken to be zero by convention.

[7] Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, 1997. ISBN 978-0-8218-0315-8

3.5 Notation Summary

The notation introduced in this chapter is collected in Table 3.1 for reference. It is used consistently throughout the remainder of this work.

Table 3.1: Notation introduced in this chapter, with the equation or section in which each symbol is defined.

Symbol	Meaning	Reference
$\mathcal{G} = (V, E)$	Graph with node set V and edge set E	(3.1)
$n,\ m$	Number of nodes and edges	§3.1
$i,\ j$	Nodes; i central, j a neighbor	§3.1
$\mathcal{N}(i)$	Neighborhood of node i	(3.2)
$\deg(i)$	Degree of node i	(3.3)
$d(i, j)$	Geodesic distance between i and j	(3.6)
π	Isomorphism bijection between node sets	(3.7)
$\mathbf{x}_i,\ d$	Node feature vector and its dimension	(3.4)
$\mathbf{X}$	Node feature matrix	(3.4)
$\mathbf{e}_{i,j},\ c$	Edge feature vector and its dimension	§3.1
w_{ij}	Scalar edge weight	§3.2
$\mathbf{A}$	Adjacency matrix	(3.5)
$(\mathbf{A}^k)_{ij}$	Walks of length k between i and j	(3.8)
$\mathbf{D}$	Degree matrix	(3.9)
$\mathbf{L}$	Graph Laplacian	(3.10)
$\mathbf{L}_{\text{sym}}$	Symmetric normalized Laplacian	(3.13)
$\mathbf{f}$	Graph signal, one real value per node	(3.11)
$\mathbf{U},\ \mathbf{\Lambda}$	Eigenvectors and eigenvalues of $\mathbf{L}$	(3.12)
$\lambda_1, \ldots, \lambda_n$	Laplacian eigenvalues, in increasing order	(3.12)

4 Neural Networks and Their Limitations on Relational Data

Contents

4.1 The Multilayer Perceptron 35
4.2 Training by Gradient Descent 36
4.3 Why the Multilayer Perceptron Fails on Graphs 38

The preceding chapters established what graphs are and the tasks defined on them. Before constructing a model that operates on graphs, it is necessary to examine the standard tool of deep learning, the multilayer perceptron. This chapter first assembles the multilayer perceptron and the procedure by which it is trained, so that it stands as a complete and well-defined object. It then attempts to apply this model to a graph and shows that the attempt fails. The reasons for that failure are not incidental: they identify precisely the properties a model must possess to operate on graphs and motivate the developments presented in the following chapters.

4.1 The Multilayer Perceptron

The *multilayer perceptron* (MLP) is the elementary feedforward neural network. It maps a fixed-size input vector to an output vector through a sequence of layers, each applying a linear transformation followed by a nonlinear function [1].

[1] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. ISBN 978-3-031-45467-7

A single layer takes a vector and produces another vector. Writing $\mathbf{h}^{(l-1)}$ for the input to layer l and $\mathbf{h}^{(l)}$ for its output, the layer computes

$$\mathbf{h}^{(l)} = \sigma\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right), \tag{4.1}$$

where $\mathbf{W}^{(l)}$ is a learnable weight matrix, $\mathbf{b}^{(l)}$ a learnable bias vector, and σ the *activation function*, a fixed nonlinear function applied elementwise. The index $l = 1, \ldots, L$ runs over the L layers of the network. The weight matrix and bias vector of each layer are the *parameters* of the

model, collectively denoted by θ, and are the quantities adjusted during training.

The activation function σ is what gives the network its expressive power. Without it, the composition of linear layers would itself be a single linear transformation, no matter how many layers were stacked. A nonlinear σ — for instance the rectified linear unit, $\sigma(x) = \max(0, x)$ — allows the network to represent nonlinear relationships between input and output [2].

A multilayer perceptron is the composition of several such layers. Taking the input vector $\mathbf{x}$ as the layer-zero representation, $\mathbf{h}^{(0)} = \mathbf{x}$, the network applies the layers in sequence,

$$f(\mathbf{x}) = f^{(L)} \circ \cdots \circ f^{(1)}(\mathbf{x}), \tag{4.2}$$

where each $f^{(l)}$ denotes the transformation of layer l defined in (4.1), and the output of the final layer is the prediction of the network. The layers between the input and the output are the *hidden layers*, and the number of layers L is the *depth* of the network. The map f takes a fixed-size vector $\mathbf{x} \in \mathbb{R}^d$ and returns an output of fixed size, determined by the dimensions of the weight matrices (Figure 4.1).

[2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. ISBN 978-0-262-03561-3

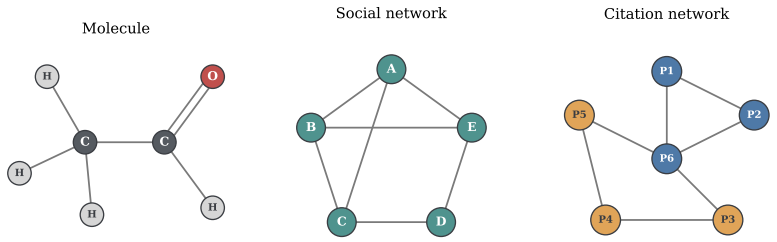

Figure 4.1: A multilayer perceptron mapping a fixed-size input vector $\mathbf{x} \in \mathbb{R}^d$ through two hidden layers to an output. Each layer applies a learnable linear transformation followed by an elementwise nonlinearity. The dimension of the input is fixed in advance by the architecture: every input the network processes must be a vector of exactly d components.

This last property is essential to what follows. The architecture fixes the size of the input in advance: the product $\mathbf{W}^{(1)}\mathbf{x}$ is defined only if the input vector $\mathbf{x}$ has the required dimension, and the network assumes that its components are arranged in a definite order. A multilayer perceptron is a function on $\mathbb{R}^d$ for a fixed d, and on nothing else.

4.2 *Training by Gradient Descent*

The parameters θ are not set by hand but learned from data, by adjusting them so that the network's predictions match ground-truth targets. This requires a measure of how wrong the predictions are and a procedure for reducing it.

The measure is a *loss function*. Given a dataset of n examples with inputs $\mathbf{x}_i$ and targets y_i, the loss aggregates the discrepancy between

each prediction $f(\mathbf{x}_i)$ and its target into a single scalar,

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(f(\mathbf{x}_i), y_i)\,, \tag{4.3}$$

where ℓ is a per-example loss, such as the squared error for regression or the cross-entropy for classification. The loss is a function of the parameters θ, since the prediction $f(\mathbf{x}_i)$ depends on them; training is the search for the θ that minimizes $\mathcal{L}(\theta)$.

That search is carried out by *gradient descent*. The gradient $\nabla_\theta \mathcal{L}$ points in the direction of steepest increase of the loss; moving the parameters in the opposite direction decreases it. Each step updates the parameters by

$$\theta \leftarrow \theta - \eta \, \nabla_\theta \mathcal{L}(\theta), \tag{4.4}$$

where the scalar $\eta > 0$ is the *learning rate*, controlling the size of each step. The gradient itself is computed by *backpropagation*, the application of the chain rule across the layers of the network [3]; it yields the derivative of the loss with respect to every parameter, and is the algorithm that makes training deep networks practical.

[3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. ISBN 978-0-262-03561-3

Computing the gradient in (4.4) over the full dataset is costly when n is large, since it requires a pass over every example for a single update. *Stochastic gradient descent* replaces the full-dataset gradient with an estimate computed on a small random subset of the data, a *minibatch* $\mathcal{B}$. The method was originally defined for a single example per step — the source of the name, as each step follows a noisy, *stochastic* estimate of the gradient rather than the exact one — but is generally applied over a minibatch, which averages the per-example gradients,

$$\nabla_\theta \mathcal{L}(\theta) \approx \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_\theta \, \ell(f(\mathbf{x}_i), y_i)\,. \tag{4.5}$$

Under uniform random sampling of examples, the minibatch gradient is an unbiased estimate of the full gradient, and each update is far cheaper, so many more updates are performed for the same computational cost. Stochastic gradient descent is the basis of essentially all modern neural network training.

The optimizer used throughout the implementations of this work is *Adam* [4], a method in the stochastic gradient descent family that adapts the effective step size for each parameter individually, using running estimates of the gradient. Its internal mechanics are not required for the developments that follow, and the reader is referred to the original work for them. What matters is that the multilayer perceptron, with its parameters, loss, and training procedure, now stands as a complete and trainable object. The remainder of the chapter asks whether that object can be applied to a graph.

[4] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015. https://arxiv.org/abs/1412.6980

4.3 Why the Multilayer Perceptron Fails on Graphs

The multilayer perceptron assembled above is a function on a fixed-size vector. A graph is not a vector. To apply the perceptron to a graph, the graph must first be converted into a vector of fixed size, and the difficulty is that no such conversion preserves what the graph encodes. This section attempts the conversion directly and examines what goes wrong; the failures that surface are the substance of the chapter.

Consider a node-level task: predicting a property of each node of a graph, as in the node classification setting of Chapter 2. There are two natural ways to present such a problem to a multilayer perceptron, and it is instructive that both fail, for different reasons.

The first approach attempts to encode the whole graph as a single input vector and feed it to the network. The structure of a graph on n nodes is carried by its adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, and its node attributes by the feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$. To form a single input vector, these matrices are flattened — their entries read out in order and laid end to end — and concatenated. This at once runs into two obstructions.

The first obstruction concerns size. The flattened adjacency matrix has n^2 entries and the flattened feature matrix has $n \times d$; the length of the resulting input vector therefore grows with the number of nodes and differs from one graph to another. But a multilayer perceptron, by (4.1), admits inputs of one fixed dimension only. A network built for a graph of one size cannot accept a graph of another. The same obstruction appears within a single graph. A finer attempt would process each node together with its neighbors, rather than flattening the whole graph at once; but nodes differ in how many neighbors they have, so this neighborhood-based input again has no fixed size (Figure 4.2). This is the problem of *variable size*: neither across graphs nor across the nodes of one graph is the number of entities fixed, whereas the multilayer perceptron demands a fixed-dimensional input.

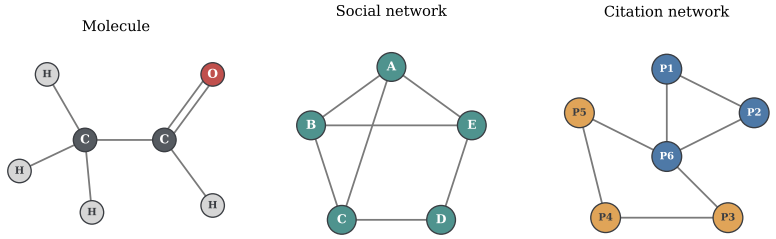

Figure 4.2: Two nodes of the same graph with neighborhoods of different sizes: the node on the left has three neighbors, the node on the right has six. A model that processes a node together with its neighbors must accept inputs of varying size, which a multilayer perceptron cannot.

The second obstruction concerns order. As established in Chapter 3, the nodes of a graph carry no intrinsic ordering; an indexing must be

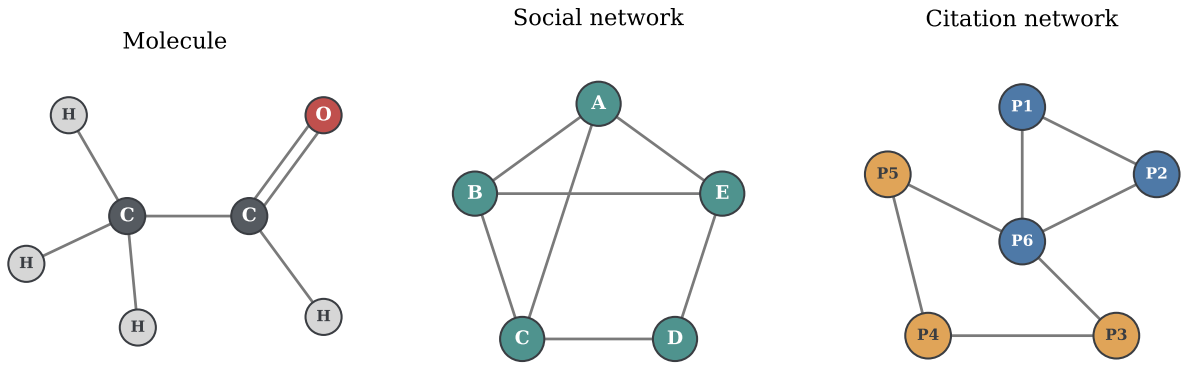

Figure 4.3: A single graph drawn twice with its nodes indexed in two different orders. Each ordering produces a different adjacency matrix, and flattening each matrix produces a different input vector. A multilayer perceptron receives the two vectors as unrelated inputs, although they encode the same graph. Neither ordering is privileged: the graph is the same object in both.

chosen to write down the adjacency matrix, and any of the $n!$ possible indexings describes the same graph. The two adjacency matrices of a graph under two different node orderings are different matrices, and flattening them yields two different vectors. A multilayer perceptron would receive these as two distinct inputs and could process them differently, even though they represent the same graph (Figure 4.3). The model would thus depend on an arbitrary choice that carries no information — the labeling of the nodes — and would have no means of recognizing that the choice is arbitrary. This is the problem of *node ordering*: the flattened representation imposes an order that the graph does not have, and the multilayer perceptron has no way to disregard it.

Faced with these obstructions, one might abandon the adjacency matrix altogether and take the second approach: present each node to the network on its own, using only its feature vector $\mathbf{x}_i$, and predict the node's property from that. This input has a fixed size d, and the ordering of the nodes no longer enters, since each node is treated separately. Both previous obstructions vanish — but at a cost that defeats the purpose. With the adjacency matrix discarded, the structure of the graph never enters the computation at all. The network processes each node as an isolated example. Yet the tasks of Chapter 2 are relational by nature: in a citation network, a paper's topic is informed by the topics of the papers it cites, and a model that sees a node in isolation discards exactly this evidence. This is the problem of *missing relational structure*: the only input that fits a multilayer perceptron is one from which the relations have been removed.

The two approaches exhaust the natural options, and they fail in opposite ways. Including the adjacency matrix preserves the structure but breaks on variable size and node ordering; excluding it restores a fixed, order-free input but discards the structure entirely. There is

no middle ground: the multilayer perceptron cannot be given a graph without either confronting a representation it cannot accept or being handed one from which the graph has been erased.

These failures reveal that the multilayer perceptron carries the wrong assumptions for relational data. Its *inductive bias* is that examples are independent and of fixed size, an assumption appropriate to vectors but false for graphs, whose entities are connected and whose representation admits no canonical order. What the analysis exposes is the assumptions a graph model must instead possess: it must accept inputs of varying size, it must be *invariant* to the ordering of the nodes, so that relabeling leaves its predictions unchanged, and it must let the structure of the graph participate in the computation. These three requirements, arrived at here by watching the multilayer perceptron fail, are the foundation on which the following chapters build a neural network suited to graphs.

5 Inductive Bias, Invariance, and Equivariance

Contents

5.1 Inductive Bias in Neural Architectures 41
5.2 The Relational Inductive Bias 42
5.3 Invariance and Equivariance 43
5.4 Permutation Invariance and Equivariance on Graphs 44

The preceding analysis in Section 4.3 introduced two concepts in passing — the *inductive bias* of a model and the requirement that a graph model be *invariant* to node ordering — without developing either. This chapter develops both. It first makes precise what an inductive bias is and why successful generalization requires one, reading the inductive biases of successful architectures as evidence that each encodes the structure of its own domain. It then identifies the inductive bias appropriate to graphs and gives the two symmetry conditions that express it formally.

5.1 Inductive Bias in Neural Architectures

A model trained on a finite set of examples must, to be useful, make predictions on examples it has never seen. The training data alone cannot determine those predictions: infinitely many functions are consistent with any finite sample. A model generalizes only because it brings assumptions of its own about which functions are plausible before any data is observed. The set of such assumptions is the *inductive bias* of the model [1]. Generalization is therefore not possible without an inductive bias, and succeeds when that bias matches the structure of the problem.

[1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. ISBN 978-0-262-03561-3

Deep learning architectures differ primarily in the inductive biases their structure encodes. Three of them recur throughout this work: the multilayer perceptron (MLP), the convolutional neural network (CNN),

and the recurrent neural network (RNN). From this point onward, these architectures are referred to exclusively by their acronyms, both in this chapter and throughout the remainder of the document.

The three differ precisely in the assumptions their structure imposes. The MLP, whose layers connect every input to every output, assumes that any input coordinate may interact with any other, that no particular structure among the coordinates is presumed, and that examples are treated independently. This is the appropriate bias when the input is an unstructured vector — such as tabular data — but the wrong bias for data with an underlying structure.

The CNN and the RNN illustrate how a well-matched bias is built into structure. The CNN, designed for data arranged on a regular grid such as the pixels of an image, applies the same local filter at every position. Two assumptions are encoded in this design. The first is *locality*: a pixel is informative about its immediate neighbors, so the filter reads only a small spatial window rather than the whole image. The second is that a pattern is meaningful regardless of where in the image it appears, so the same filter is reused at every position; a feature shifted across the grid produces the same feature shifted in the output. These assumptions make the CNN well suited to natural images. The RNN, designed for sequences, encodes a different bias: it processes elements in order, applying the same transition at every step. This assumes that the data has a meaningful sequential order and that the rule relating one element to the next remains the same throughout the sequence. These assumptions are appropriate for text and time series.

The same principle appears in all three. Each architecture succeeds when its inductive bias matches the structure of the data, and the match is built into how the architecture is wired rather than learned from data. The MLP assumes no particular structure; the CNN assumes a grid with local, position-independent patterns; the RNN assumes an ordered sequence with a fixed step-to-step rule. The question this chapter must answer follows immediately: what is the structure of a graph, and what inductive bias matches that structure.

5.2 *The Relational Inductive Bias*

The defining structure of a graph is relational. An entity is not described in isolation but through its connections to other entities: in a citation network the subject of a paper is reflected in the papers it cites, and in a molecule the role of an atom is determined by the atoms bonded to it. The inductive bias appropriate to graphs is therefore the assumption that a node is characterized by its neighborhood — that the information relevant to a node resides in the nodes adjacent to it and in the structure connecting them. This assumption is the *relational inductive bias* [2].

[2] Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Caglar Gulcehre, Francis Song, Andrew Ballard, Justin Gilmer, George Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matt Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. Relational inductive biases, deep learning, and graph networks. arXiv:1806.01261 [cs.LG], 2018. URL https://arxiv.org/abs/1806.01261

The relational inductive bias is the graph counterpart of the locality that the CNN encodes, and the comparison is exact in one respect and instructive in its difference. Both assume that an entity is informed by what lies near it rather than by everything at once, and both therefore read from a local region rather than the whole input. The difference lies in what counts as local. On a grid, locality is geometric: a pixel's neighbors are the pixels at fixed offsets around it, the same offsets everywhere, so the notion of a local window is defined once and applied uniformly. On a graph there are no fixed offsets and no uniform window. A node's neighborhood is whatever the edges make it, varying in size from one node to the next. The relational inductive bias retains the principle of locality while discarding the regular geometry that made it simple to express on a grid; locality on a graph is defined by adjacency, not by position.

Adjacency, however, fixes only which nodes are neighbors, not the order in which they are listed. A neighborhood is a set, and a set has no first element. The relational inductive bias says where a node's information comes from; it does not, and must not, depend on the order in which that information is enumerated, because the graph supplies no such order. Making this requirement precise calls for the concepts of the next two sections.

5·3 *Invariance and Equivariance*

The requirement that a model not depend on an arbitrary choice in its input is made precise by the notions of *invariance* and *equivariance*. Both describe how a function responds to transformations of its input, and they are defined for any function and any transformation before being specialized to graphs.

Let f be a function and let g denote a transformation applied to its input. The function f is *invariant* to g when transforming the input leaves the output unchanged:

$$f(g \cdot \mathbf{x}) = f(\mathbf{x}). \tag{5.1}$$

The transformation is applied to the input, and the output does not move. The function f is *equivariant* to g when transforming the input transforms the output in the same way:

$$f(g \cdot \mathbf{x}) = g \cdot f(\mathbf{x}). \tag{5.2}$$

The transformation is applied to the input, and here the output moves in step with the input.

The distinction is between discarding a transformation and preserving it. Invariance identifies transformed and untransformed

inputs by mapping them to the same output, so the transformation is discarded by f. Equivariance preserves the effect of the transformation: it survives the application of f and acts on the result, so applying f and then transforming gives the same answer as transforming and then applying f. Which of the two is required is not a matter of preference but of the task: when the quantity to be predicted is itself unaffected by the transformation, f must be invariant; when the quantity to be predicted is attached to parts of the input that the transformation moves, f must be equivariant, so that the prediction moves with the part it describes.

5·4 *Permutation Invariance and Equivariance on Graphs*

The transformation that matters for graphs is the reordering of nodes. As established in Chapter 3, the nodes of a graph carry no intrinsic order; an indexing must be chosen to write the adjacency matrix $\mathbf{A}$ and the feature matrix $\mathbf{X}$, and any of the $n!$ possible indexings describes the same graph. A reordering of the nodes is represented by a *permutation matrix* $\mathbf{P} \in \{0,1\}^{n \times n}$, a square matrix with exactly one entry equal to 1 in each row and each column and zeros elsewhere. An entry $P_{ij} = 1$ indicates that the node originally at index j is mapped to the new index i. Multiplying by $\mathbf{P}$ rearranges the rows of a matrix according to this relabeling. A permutation matrix is orthogonal,

$$\mathbf{P}\mathbf{P}^\top = \mathbf{I}, \tag{5.3}$$

which states that the reordering is invertible and that its inverse is its transpose: the rearrangement can always be undone.

Applying the reordering $\mathbf{P}$ to a graph acts on both of its matrices. The feature matrix becomes $\mathbf{PX}$, which moves each node's feature vector to the row assigned by the reordering. The adjacency matrix becomes $\mathbf{PAP}^\top$, with $\mathbf{P}$ reordering its rows and $\mathbf{P}^\top$ its columns, so that the entry recording an edge between two nodes follows them to their new positions. The pair $(\mathbf{PAP}^\top, \mathbf{PX})$ is the same graph as $(\mathbf{A}, \mathbf{X})$ under a different indexing; no edge has been added or removed, and no feature has changed, only the node ordering.

A function defined on graphs must respect this fact, and which condition it must satisfy depends on the level of the task. For a task whose output describes the graph as a whole — a single prediction for the entire graph, as in graph classification — the output must not depend on the indexing at all. Such a function f must be *permutation invariant*:

$$f(\mathbf{PAP}^\top, \mathbf{PX}) = f(\mathbf{A}, \mathbf{X}). \tag{5.4}$$

Reordering the nodes leaves the graph-level prediction unchanged. For a task whose output is attached to the individual nodes — one

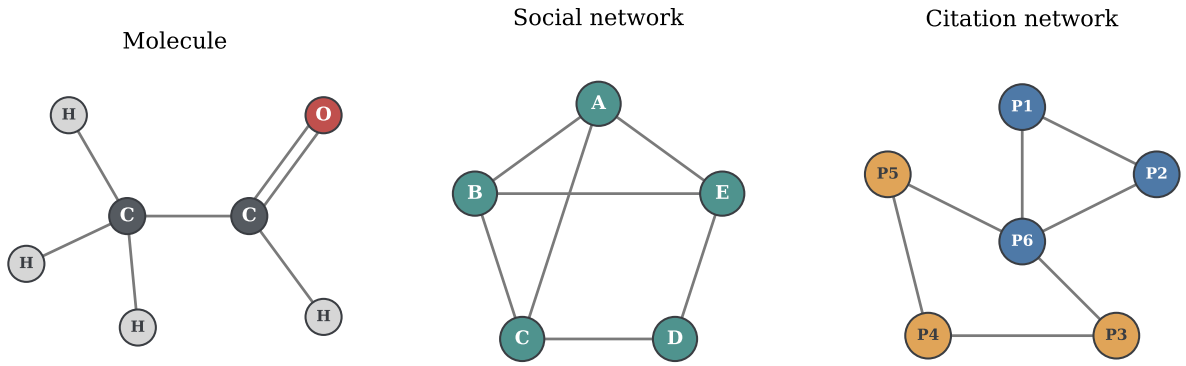

Figure 5.1: A single graph shown under two node orderings related by a permutation $\mathbf{P}$. Left: the original indexing, with a per-node output attached to each node and a single graph-level output computed from the whole graph. Right: the reordered indexing, with the graph-level output unchanged (invariance), while the per-node outputs appear in the permuted positions (equivariance).

prediction per node, as in node classification — the output must follow the nodes when they are reordered. In this case, f must be *permutation equivariant*:

$$f(\mathbf{PAP}^\top, \mathbf{PX}) = \mathbf{P}\, f(\mathbf{A}, \mathbf{X}). \tag{5.5}$$

Reordering the nodes reorders the per-node outputs in exactly the same way: the prediction for a given node is the same prediction, carried to that node's new position [3]. Figure 5.1 illustrates the two conditions on a single graph drawn under two orderings.

These two conditions, together with the relational inductive bias of Section 5.2, determine what a layer operating on graphs must be. The relational inductive bias fixes where each node's information comes from. Permutation equivariance fixes how the layer must treat the indexing. This is the design the next chapter develops: the message-passing framework gives these requirements a concrete and general form.

[3] William L. Hamilton. *Graph Representation Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool, 2020. ISBN 978-1-68173-965-6

6 The Message-Passing Framework

Contents

6.1 From Relational Bias to Neighborhood Aggregation **47**
6.2 The Message-Passing Layer **49**
6.2.1 The Message Function 49
6.2.2 The Aggregation Function 50
6.2.3 The Update Function 50
6.2.4 The General Equation of a Message Passing Layer 51
6.3 Stacking Layers: Receptive Field and Depth . . **52**
6.4 Transductive and Inductive Learning **53**
6.5 Expressive Power and the 1-WL Bound **54**
6.6 The Design Space **55**
6.6.1 Flexibility of the Framework 56
6.6.2 The MLP as a Degenerate Case 56
6.6.3 Established Architectures as Instances 57

A graph model must handle inputs of varying size, remain independent of node ordering, and incorporate graph structure into the computation. These requirements are captured by the relational inductive bias, together with the conditions of permutation invariance and equivariance. This chapter constructs a layer that satisfies these properties by design, assembling it from three component functions, examining the effects of stacking such layers, and interpreting the resulting formulation as a general design space for GNNs.

6.1 From Relational Bias to Neighborhood Aggregation

The relational inductive bias of Section 5.2 asserts that a node is characterized by its neighborhood. This fixes where a node's representation must come from: not from the node alone, but from the node together with the nodes adjacent to it. A layer that respects the bias therefore computes a node's representation by *aggregating* the information held by its neighbors. This is the operative idea.

The question is how to perform that aggregation while meeting the requirements the previous chapters imposed.

The CNN offers an instructive point of comparison, because it faces the same problem on a fixed-size domain and solves it in a way that does not transfer directly [1]. A convolutional layer reads each position together with a fixed window of surrounding positions and applies the same filter at every position. Two aspects of this design are worth isolating. The window is of fixed size and fixed shape, so the filter always reads the same number of inputs arranged in the same way; and the filter is reused everywhere, so a single set of parameters serves the whole input. A graph layer should follow the same instinct — read a node together with its local region rather than the whole graph — but the neighborhood structure of a graph differs fundamentally. A neighborhood is not a fixed window: different nodes have different numbers of neighbors, so the operation must accept a variable number of inputs. And a neighborhood has no fixed arrangement: it is a set, with no first or last element, so the operation must not depend on the order in which the neighbors are listed.

[1] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. ISBN 978-3-031-45467-7

These two obstacles are met by two ingredients acting together, and the distinction between what each contributes is worth drawing precisely, since it is easy to attribute to one what belongs to the other. The first ingredient is the aggregation itself. An operation that combines a neighborhood of arbitrary size into a single vector resolves the problem of variable size directly: its output has a fixed dimension regardless of how many neighbors entered it. If, in addition, that operation is insensitive to the order of its inputs, it also discards the arbitrary ordering, and the requirement of Section 5.4 is satisfied at the level of the neighborhood. The aggregation, then, handles the variable size and supplies invariance to the ordering of neighbors.

The second ingredient is *parameter sharing*: the same functions, with the same parameters, are applied at every node of the graph, exactly as a single filter is reused at every position of an image. Its contribution is distinct from that of the aggregation. Because the identical computation is performed at every node, a relabeling of the nodes does not change the computation applied to them. Combined with an order-insensitive aggregation, it allows the desired permutation properties to be built into the layer.

Parameter sharing also keeps the number of parameters independent of the number of nodes. The same layer can therefore be applied to graphs of arbitrary size, which both improves scalability and permits a model trained on one graph to be applied to another [2]. The two ingredients are therefore complementary — the parameter sharing guarantees that the same computation is applied to every node, and the aggregation collapses a variable, unordered neighborhood into a fixed

[2] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. ISBN 978-3-031-45467-7

vector — and together they turn the relational inductive bias into an operation that meets the requirements of the two preceding chapters.

When a node's representation is formed by combining information from its neighborhood, through repeated aggregation and shared transformations, connected nodes tend to acquire increasingly similar representations. This behavior aligns the layer with the *homophily* assumption introduced in Section 3.3, namely that adjacent nodes are likely to be similar. For many graphs this bias is appropriate and is precisely what makes the approach effective. When neighboring nodes carry information that differs systematically from that of the central node, however, treating their contributions uniformly may become a limitation, a question taken up in Chapter **??**.

6.2 The Message-Passing Layer

The ideas developed in the previous section can now be stated formally. A layer of the kind described takes the representations of all nodes and produces new representations for each node, each formed from the node and its neighborhood. This section assembles such a layer from three functions — one that forms a message from a neighbor, one that aggregates the messages of a neighborhood, and one that updates a node from its aggregated neighborhood — and states the general equation they compose into. The unification of these three functions into a single framework, under which many previously separate graph models appear as instances, is due to Gilmer et al. [3]; the exposition here follows the formulation used throughout this work.

[3] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1263–1272, 2017. https://arxiv.org/abs/1704.01212

The object the layer transforms is the *hidden representation* of a node. Writing $\mathbf{h}_i^{(k)} \in \mathbb{R}^{d'}$ for the representation of node i after layer k, the layer maps the node representations $\{\mathbf{h}_i^{(k-1)}\}$ to updated representations $\{\mathbf{h}_i^{(k)}\}$. The representation at layer zero is initialized to the input features, $\mathbf{h}_i^{(0)} = \mathbf{x}_i$, so the first layer reads the raw node attributes and each subsequent layer reads the output of the one before. The representations produced by the intermediate layers are therefore hidden. As with the feature matrix $\mathbf{X}$, whose row i is the feature vector $\mathbf{x}_i$, the hidden representations of all nodes are collected into a matrix $\mathbf{H}^{(k)} \in \mathbb{R}^{n \times d'}$ whose row i is $\mathbf{h}_i^{(k)}$; the matrix form is used when a layer is written as a single operation on the whole graph.

6.2.1 The Message Function

The first function specifies what a neighbor contributes. For a node i and a neighbor $j \in \mathcal{N}(i)$, the *message function* $\phi^{(k)}$ produces a vector — the message sent from j to i — from the representations available at the two endpoints of the edge and, where present, the feature of the edge

itself:

$$\mathbf{m}_{j\to i}^{(k)} = \phi^{(k)}\left(\mathbf{h}_i^{(k-1)}, \mathbf{h}_j^{(k-1)}, \mathbf{e}_{j,i}\right). \tag{6.1}$$

The function $\phi^{(k)}$ is a learnable map whose output $\mathbf{m}_{j\to i}^{(k)}$ is the message; its arguments are the representation of the receiving node i, the representation of the sending neighbor j, and the feature $\mathbf{e}_{j,i}$ of the edge from j to i. Not every argument need be used: a message may be formed from the neighbor alone, from the neighbor together with the central node, or from either of these together with the edge feature. The general form admits all three sources — the neighbor, the central node, and the edge — and a particular architecture selects among them. The edge feature is retained in the general form because the framework accommodates it, and because it is the point at which edge information would enter when available.

6.2.2 *The Aggregation Function*

The message function produces one message for each neighbor of i. Since a node may have any number of neighbors, these messages form a collection of variable size, and they must be combined into a single vector before the node can be updated. The *aggregation function* $\bigoplus$ performs this combination over the neighborhood:

$$\mathbf{a}_i^{(k)} = \bigoplus_{j\in\mathcal{N}(i)} \mathbf{m}_{j\to i}^{(k)}. \tag{6.2}$$

The aggregated vector $\mathbf{a}_i^{(k)}$ summarizes the entire neighborhood of i in a single vector of fixed dimension, whatever the number of neighbors, which is what allows the layer to accept neighborhoods of varying size. The neighborhood $\mathcal{N}(i)$ is a set, and the messages carry no intrinsic order; the result of combining them must therefore not depend on the order in which they are taken. The aggregation function must be invariant to the ordering of its inputs. The operations commonly used — the sum, the mean, and the maximum, taken coordinatewise — all satisfy this condition, and any of them yields a well-defined aggregation.

6.2.3 *The Update Function*

The aggregated neighborhood does not by itself determine the node's new representation. Although information about the node itself may already enter through the message function, the layer still requires a mechanism that combines the neighborhood summary with the node's own prior representation. The *update function* $\gamma^{(k)}$ combines the two, producing the new representation from the node's previous

representation and the aggregate of its neighborhood:

$$\mathbf{h}_i^{(k)} = \gamma^{(k)}\left(\mathbf{h}_i^{(k-1)}, \mathbf{a}_i^{(k)}\right). \tag{6.3}$$

The function $\gamma^{(k)}$ is a learnable map that takes the node's own prior representation $\mathbf{h}_i^{(k-1)}$ together with the aggregated message $\mathbf{a}_i^{(k)}$ and returns the updated representation $\mathbf{h}_i^{(k)}$. Retaining the node's prior state alongside the neighborhood is what prevents the node's own information from being discarded; the balance between the two is a matter of how $\gamma^{(k)}$ is chosen, and ranges from a simple sum of the two arguments to a learnable combination, a question taken up with the design space in Section 6.6.

6.2.4 *The General Equation of a Message Passing Layer*

Composing the three functions gives the general form of a message-passing layer. Substituting the message function (6.1) into the aggregation (6.2), and the result into the update (6.3), the representation of node i at layer k is

$$\mathbf{h}_i^{(k)} = \gamma^{(k)}\left(\mathbf{h}_i^{(k-1)}, \bigoplus_{j \in \mathcal{N}(i)} \phi^{(k)}\left(\mathbf{h}_i^{(k-1)}, \mathbf{h}_j^{(k-1)}, \mathbf{e}_{j,i}\right)\right). \tag{6.4}$$

This equation defines the message-passing framework (Figure 6.1), and the architectures of the following chapters are obtained by choosing the three functions $\phi^{(k)}$, $\bigoplus$, and $\gamma^{(k)}$. Where any of these functions applies a learnable linear transformation, the transformation is written with a weight matrix $\mathbf{W}$, and an associated bias is always understood to be present even where it is not written: the bias is frequently absorbed into the weight notation or omitted for brevity, and its omission from a particular equation never signifies its absence. Hamilton expresses the same construction with operators named AGGREGATE and UPDATE, in place of $\bigoplus$ and $\gamma^{(k)}$; the two formulations describe the same computation [4].

The layer assembled in this way meets the permutation conditions of Section 5.4: the message function (6.1) is applied identically to every edge, the aggregation (6.2) is invariant to the order of the neighbors, and the update (6.3) is applied identically to every node. Relabeling the nodes by a permutation therefore does not change any node's computed representation; it only moves that representation to the node's new index. In the matrix notation of Section 5.4, a layer that sends $(\mathbf{A}, \mathbf{H}^{(k-1)})$ to $\mathbf{H}^{(k)}$ sends the relabeled graph $(\mathbf{PAP}^\top, \mathbf{PH}^{(k-1)})$ to $\mathbf{PH}^{(k)}$, which is the condition of permutation equivariance in (5.5). The per-node output follows the nodes when they are reordered, as a node-level task requires. A task defined on the whole graph requires instead

[4] William L. Hamilton. *Graph Representation Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool, 2020. ISBN 978-1-68173-965-6

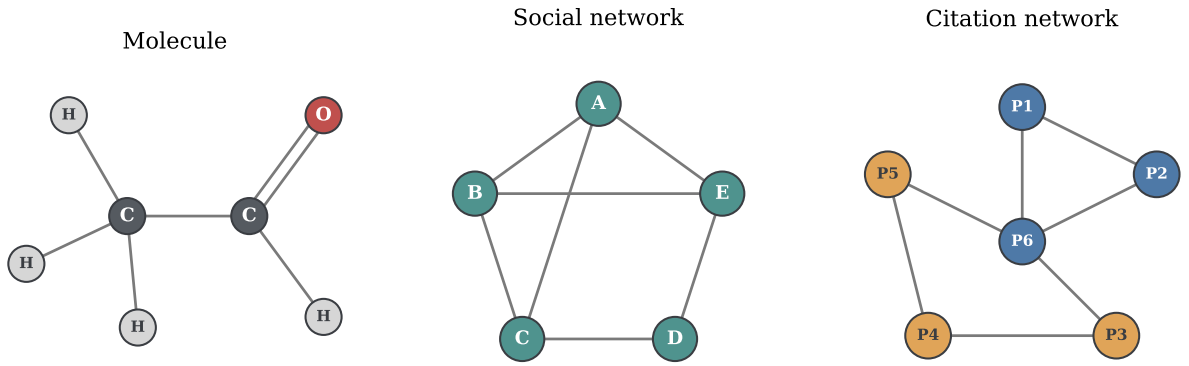

Figure 6.1: One message-passing layer applied to a central node i. For each neighbor $j \in \mathcal{N}(i)$, the message function $\phi^{(k)}$ forms a message from the representations of j and i and, where present, the edge feature. The aggregation $\bigoplus$ combines the neighbors' messages into a single vector, invariant to their order. The update function $\gamma^{(k)}$ combines this aggregate with the node's own previous representation to produce $\mathbf{h}_i^{(k)}$.

a single output unchanged by relabeling — the permutation invariance of (5.4) — and this is obtained by reducing the per-node representations to one vector with a further order-insensitive operation; that operation, called *readout*, is the one used by graph-level architectures, and is developed in Chapter 7.

6.3 Stacking Layers: Receptive Field and Depth

A single message-passing layer updates each node from its immediate neighbors. Applying the layer once therefore lets each node incorporate information from one step away. To reach further, layers are stacked: the output representations of one layer become the input representations of the next, and a network of K layers applies the update (6.4) K times in succession, with $k = 1, \dots, K$. Each layer carries its own functions $\phi^{(k)}, \gamma^{(k)}$ and its own parameters; the parameters are shared across the nodes within a layer, as Section 6.1 established, but not across layers, so the transformation applied at depth k differs from the one applied at depth $k-1$.

Stacking has a definite effect on how far information can propagate into a node's representation, and the effect compounds with depth. After one layer, $\mathbf{h}_i^{(1)}$ depends on the nodes in $\mathcal{N}(i)$, the neighbors at distance one. After two layers, $\mathbf{h}_i^{(2)}$ is computed from the first-layer representations of those neighbors, and each of those already depends on its own neighbors; the representation of i thus comes to depend on the nodes within distance two. In general, after k layers the representation $\mathbf{h}_i^{(k)}$ depends on every node within geodesic distance k of i — the set of nodes j with $d(i,j) \leq k$, the distance being the one defined in Chapter 3. This is the *receptive field* of node i at depth k: the region of the graph that can influence the node's representation.

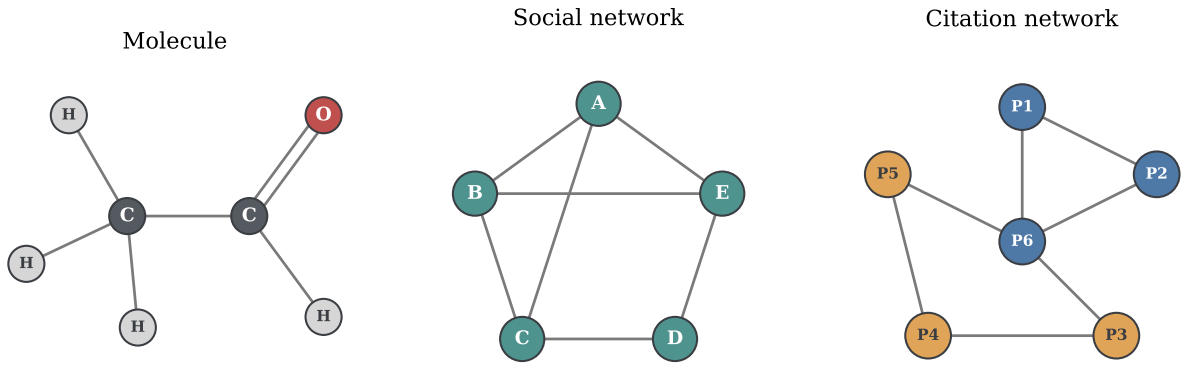

Figure 6.2: Left: a target node in the input graph. Right: the computation that produces its representation, unrolled over message-passing layers. The first layer aggregates messages from the node's immediate neighbors; the second aggregates messages from the neighbors of those neighbors, and so on. After k layers the representation of the target node depends on every node within k hops of it, the receptive field at depth k.

Each additional layer expands the receptive field by one *hop*, so the depth of the network sets the radius of the neighborhood that each node ultimately incorporates (Figure 6.2).

The depth of the network thus controls the extent of the graph that each node's representation can incorporate, and this is the quantity a designer sets in choosing the number of layers. The final representations $\mathbf{h}_i^{(K)}$, produced after the last of the K layers, are the *embeddings* of the nodes: the learned vectors that summarize each node together with its K-hop context, and that serve as the input to whatever prediction the task requires — a classifier applied per node, a score computed on pairs of nodes, or a reduction over all nodes to a single vector for the graph.

6.4 *Transductive and Inductive Learning*

The way a message-passing network is trained and evaluated depends on what portion of the graph data is available during training. In practice, the two task levels introduced in Chapter 2 commonly give rise to two distinct learning settings. The distinction concerns what the network has access to during training and what is held back for evaluation, and it follows directly from the fact that a node's representation is computed from its neighborhood.

Consider first a task defined at the node level, in which each node carries a label and the goal is to predict the labels of some nodes from those of others. As in any supervised problem, the nodes are partitioned into a training set, whose labels are used to fit the model, and an evaluation set, whose labels are withheld and used to measure generalization. The graph structure, however, prevents these two sets from being separated cleanly. When the network computes the representation of a training node, it aggregates the representations

of that node's neighbors, and some of those neighbors belong to the evaluation set. Their features and their position in the graph therefore enter the computation performed during training, even though their labels do not. This is the *transductive* setting. In node-classification problems it is often described as a form of *semi-supervised learning*, since the labels of only a subset of the nodes are available during training.

A task defined at the graph level admits a cleaner separation. Here the dataset is a collection of whole graphs, each carrying a single label, and the partition into training and evaluation sets is a partition of the graphs. A graph held out for evaluation is absent from training in its entirety — its nodes, its edges, and its label are all unseen — because the unit of the partition is the whole graph rather than a node within a shared structure. This is the *inductive* setting, and it is the one that arises in the usual sense of training on some examples and generalizing to others never seen before.

What enables the inductive setting is the parameter sharing of Section 6.1. Because the functions $\phi^{(k)}$ and $\gamma^{(k)}$ are shared across nodes and carry no dependence on the identity or the number of nodes in any particular graph, a network fitted on one set of graphs is immediately applicable to another: the same functions compute representations on a graph the network has never encountered. The same property that decouples the parameter count from graph size is thus what allows a message-passing network to generalize across graphs.

6.5 *Expressive Power and the 1-WL Bound*

The framework constructed so far guarantees that a message-passing layer respects the permutation conditions of a graph, but it says nothing about how much the resulting representations can distinguish. A natural way to measure the capability of a graph model is to ask which graphs it can tell apart: given two graphs that are not isomorphic, does the network assign them different representations, or does it map structurally different graphs to the same output? This is the question of *expressive power*, and it is the property that governs how well a model can serve graph-level tasks, where distinguishing one graph from another is the whole of what is required. Graph isomorphism, defined in Chapter 3, is the precise notion of two graphs being the same up to a relabeling; expressive power asks how close a network comes to distinguishing every pair of graphs that are genuinely different.

There is a classical procedure for distinguishing non-isomorphic graphs that was introduced before GNNs and turns out to mirror message passing closely. The one-dimensional Weisfeiler–Leman test, abbreviated 1-WL, assigns a color to every node and refines these colors iteratively. Initially all nodes share a common color. At each round, a

node's new color is determined by its current color together with the collection of colors of its neighbors; nodes that have the same color and whose neighbors have matching collections of colors keep a common color, while any difference in the surrounding colors splits them apart. The refinement continues until the partition of nodes into colors no longer changes. Two graphs for which the refinement process produces different colorings are certainly not isomorphic.

The correspondence with message passing is structural and immediate. A round of 1-WL updates a node's color from its own color and the colors of its neighbors; a message-passing layer updates a node's representation from its own representation and the aggregated representations of its neighbors. The color plays the role of the hidden representation, and the refinement step plays the role of one layer. This parallel was established by Morris et al. [5], and it has a consequence that bounds the framework from above: a message-passing network can distinguish two graphs only if 1-WL assigns them different colorings. No network built from the general equation (6.4) can separate a pair of graphs that 1-WL fails to separate. The expressive power of message passing is thus bounded above by 1-WL.

This bound is a ceiling: a particular architecture may fall short of it, depending on how its aggregation and update functions are chosen. Reaching the ceiling — matching the full discriminative power of 1-WL rather than settling below it — requires the aggregation to preserve enough information about the neighborhood that distinct neighborhoods always yield distinct aggregated vectors. This requirement strongly favors aggregation functions that preserve as much neighborhood information as possible, and motivates the architecture developed for graph-level tasks in Chapter 7. The question of what the 1-WL ceiling excludes — the structurally different graphs that no message-passing network can tell apart — is a limitation of the framework and is taken up in Chapter **??**.

[5] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and Leman go neural: Higher-order graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 4602–4609, 2019. DOI: 10.1609/aaai.v33i01.33014602

6.6 *The Design Space*

The general equation (6.4) is not a single model but a template. It fixes the form of a layer — a message formed per neighbor, an aggregation over the neighborhood, an update of the node — and requires only that the three functions respect the permutation conditions; it does not fix what those functions are. Every choice of $\phi^{(k)}$, $\bigoplus$, and $\gamma^{(k)}$ that meets the conditions produces a valid layer, and the space of such choices is the design space of the framework. The architectures studied in this work are points in this space, and reading the equation as a space of possibilities rather than a fixed formula is what makes their differences understandable.

6.6.1 Flexibility of the Framework

The flexibility of the equation can be seen function by function. The message function $\phi^{(k)}$ may be as simple as a linear transformation of the neighbor's representation, or as involved as an MLP applied to the neighbor, the central node, and the edge feature together; it may apply one weight matrix to the central node and a different one to its neighbors, allowing a node to treat its own information differently from that of its neighborhood. The aggregation $\bigoplus$ may be any order-insensitive operation, with the sum, mean, and maximum being the common choices, each preserving a different aspect of the neighborhood. The update $\gamma^{(k)}$ may be a linear map followed by a nonlinearity, an MLP, or a rule that concatenates the node's prior representation with the aggregated neighborhood, $\mathbf{h}_i^{(k-1)} \oplus \mathbf{a}_i^{(k)}$, before transforming the result, so that the two are kept distinct rather than summed. Each such decision is a coordinate in the design space, and a complete architecture is a setting of all of them.

The flexibility extends beyond the choice of functions to the very object whose representation is learned. The equation forms a node's representation from messages along its edges, but nothing in its structure requires the unit of representation to be a node. A layer that built its messages from edge features and aggregated over adjacent edges would, by the same mechanism, learn representations of the edges themselves; the message, aggregation, and update functions retain their roles, and only the object they act on changes. Edge-centered architectures found in the literature are instances of the same framework, reoriented from nodes to edges. This generality indicates that message passing is not tied to node representations specifically, but is a template for computing on graph elements through local aggregation.

6.6.2 The MLP as a Degenerate Case

One point in the design space recovers a familiar model and locates the framework relative to it. Suppose the message function is discarded and the update depends on the node alone, so that the layer computes $\mathbf{h}_i^{(k)} = \gamma^{(k)}(\mathbf{h}_i^{(k-1)})$ with no contribution from the neighborhood. The graph structure plays no part; each node is transformed in isolation, and the layer reduces to an MLP applied independently to every node. The MLP is therefore the degenerate point of the design space at which the neighborhood is ignored — precisely the model that Chapter 4 found unable to use graph structure.

Read in the other direction, this shows that a message-passing network is a generalization of the MLP, with the MLP appearing as a

particular instance of the framework. The structure that Chapter 4 identified as missing is supplied by exactly the terms that the degenerate case removes, which places the message-passing framework and the MLP at two ends of a single construction.

6.6.3 *Established Architectures as Instances*

The design space is not merely a catalog of possibilities; it is populated by architectures that have been proposed, studied, and applied, each fixing the three functions in a particular way to serve a particular purpose. The recognition that these architectures, developed separately, are instances of one framework is itself the contribution of the message-passing formulation [6]: models that appear distinct on the surface differ only in their choice of message, aggregation, and update.

This work examines three such instances, one representative of each task level introduced in Chapter 2: a node-level architecture built on the assumption that adjacent nodes are similar and whose aggregation smooths each node toward its neighborhood; an edge-level architecture that uses node embeddings to score the plausibility of a link between two nodes; and a graph-level architecture whose aggregation is chosen to reach the 1-WL ceiling of Section 6.5, maximizing the network's power to distinguish whole graphs. Each is the subject of the following chapter, where the general framework of this chapter is given a concrete form.

[6] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1263–1272, 2017. https://arxiv.org/abs/1704.01212

7 Representative Architectures for Node-, Edge-, and Graph-Level Tasks

Contents

7.1 Node-Level Learning **59**
7.1.1 From Embeddings to Predictions . . . 60
7.1.2 The Graph Convolutional Network . 60
7.2 Edge-Level Learning **62**
7.2.1 The Encoder–Decoder Framework . . 63
7.2.2 The Graph Autoencoder 64
7.3 Graph-Level Learning **65**
7.3.1 Pooling and Readout 65
7.3.2 The Graph Isomorphism Network . . 66

The message-passing framework of Chapter 6 fixes the form of a layer without fixing its content: a message per neighbor, an order-insensitive aggregation, and an update, with the three functions left open. The design space of Section 6.6 is the set of admissible choices for them. This chapter selects three points in that space, one for each of the task levels introduced in Chapter 2. Each architecture is presented as an instance of the general equation (6.4), and each is paired with the elements its task demands beyond the message-passing layer itself: how a per-node prediction is formed for node-level tasks, how a pair of nodes is scored for edge-level tasks, and how the per-node representations are reduced to a single vector for graph-level tasks.

7.1 Node-Level Learning

A node-level task assigns a prediction to each node of a single graph. The machinery this requires beyond a message-passing layer is small, because most of it is already in place. Chapter 6 established that stacking K layers produces, for every node, an embedding $\mathbf{h}_i^{(K)}$ that summarizes the node together with its K-hop context, and that the per-node output of an equivariant network follows the nodes under

relabeling, as a node-level task requires. Chapter 6 also established the transductive setting in which such a task is trained. What remains is to specify how a prediction is read off from each embedding, and then to fix the three functions of the layer.

7.1.1 From Embeddings to Predictions

In node classification each node carries a label drawn from a fixed set of classes, and the goal is to predict the labels of the unlabeled nodes. The embedding $\mathbf{h}_i^{(K)}$ produced by the final message-passing layer is the representation on which this prediction is based. A prediction is obtained by applying a *classifier* to each node's embedding: a function shared across nodes that maps an embedding to a distribution over the classes. The standard choice maps the embedding to one score per class through a learnable linear transformation and normalizes these scores to a distribution,

$$\hat{\mathbf{y}}_i = \operatorname{softmax}\left(\mathbf{W}\,\mathbf{h}_i^{(K)}\right), \tag{7.1}$$

where $\mathbf{W}$ is a weight matrix with one row per class, the bias understood as in Section 6.2.4, and softmax the function that turns a vector of scores into nonnegative entries summing to one. The entry of $\hat{\mathbf{y}}_i$ with largest value is the predicted class of node i. Because the classifier is shared across nodes and the embeddings are produced equivariantly, the prediction attached to a node is unchanged by a relabeling of the graph, that is, it moves with the node, exactly as the task requires.

7.1.2 The Graph Convolutional Network

The *Graph Convolutional Network (GCN)* is the instance of the message-passing framework obtained by taking the aggregation to be a normalized sum of the neighbors' representations, with no per-neighbor weighting beyond that normalization [1]. It is presented here first in the per-node form that exhibits it as an instance of the general equation, and then in the matrix form that connects it to the spectral perspective of Section 3.4.

To exhibit GCN as a message-passing layer, fix the three functions of the general equation (6.4) as follows. The message from a neighbor j is its representation, scaled by a coefficient that depends only on the degrees of the two endpoints. There is no separate weighting of the central node, and edge features play no role. The aggregation is the sum. The update applies a single shared linear transformation to the aggregated neighborhood, followed by a nonlinearity. Writing this out,

[1] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the 5th International Conference on Learning Representations*, 2017. https://arxiv.org/abs/1609.02907

the representation of node i at layer k is

$$\mathbf{h}_i^{(k)} = \sigma\left(\mathbf{W}^{(k)} \sum_{j \in \mathcal{N}(i)\cup\{i\}} \frac{1}{\sqrt{\widetilde{\deg}(i)}\sqrt{\widetilde{\deg}(j)}}\, \mathbf{h}_j^{(k-1)}\right), \tag{7.2}$$

where $\mathbf{W}^{(k)}$ is the layer's shared weight matrix, σ is an elementwise nonlinearity, and $\widetilde{\deg}(i) = \deg(i) + 1$ is the degree of node i after the addition of the self-loop. Two features of this form distinguish GCN from the general layer. First, the sum runs over $\mathcal{N}(i) \cup \{i\}$ rather than $\mathcal{N}(i)$: the node includes itself among the terms aggregated as a self-loop. This replaces the separate update over $\mathbf{h}_i^{(k-1)}$ used in the general equation; in GCN the node and its neighbors are treated by one and the same transformation, the distinction between message and update collapsing into a single aggregation over the closed neighborhood. Second, each term carries the coefficient $1/\sqrt{\widetilde{\deg}(i)}\sqrt{\widetilde{\deg}(j)}$, which scales a neighbor's contribution down when either endpoint has high degree. This *symmetric normalization* keeps the magnitude of the aggregated vector from growing with the number of neighbors, and weights a neighbor by how distinctive the connection is rather than treating all neighbors of a high-degree node alike.

The aggregation (7.2) is a sum over a set and is therefore invariant to the order of the neighbors, and the weight matrix $\mathbf{W}^{(k)}$ is shared across all nodes. The layer thus meets the permutation conditions of Section 5.4 by the same argument as the general layer.

When the same operation is written for all nodes at once, the per-node sum (7.2) becomes a single matrix product. The self-connection and the degree normalization are first absorbed into two matrices. Adding a self-loop at every node gives

$$\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}, \qquad \tilde{D}_{ii} = \sum_j \tilde{A}_{ij}, \tag{7.3}$$

where $\tilde{\mathbf{A}}$ is the adjacency matrix with self-loops and $\tilde{\mathbf{D}}$ is its degree matrix, the diagonal matrix holding the row sums of $\tilde{\mathbf{A}}$. The substitution of $\tilde{\mathbf{A}}$ for $\mathbf{A}$ and of $\tilde{\mathbf{D}}$ for the degree matrix of $\mathbf{A}$ is the *renormalization trick*: it folds the self-connection of Equation (7.2) into the propagation and, as the spectral reading below makes precise, keeps the operator's spectrum within a stable range. With these matrices the layer is

$$\mathbf{H}^{(k)} = \sigma\left(\tilde{\mathbf{D}}^{-1/2}\,\tilde{\mathbf{A}}\,\tilde{\mathbf{D}}^{-1/2}\,\mathbf{H}^{(k-1)}\,\mathbf{W}^{(k)}\right), \tag{7.4}$$

where $\mathbf{H}^{(k)}$ is the matrix of node representations of Section 6.2. The factors $\tilde{\mathbf{D}}^{-1/2}$ on each side of $\tilde{\mathbf{A}}$ supply the symmetric normalization, reproducing the coefficient $1/\sqrt{\widetilde{\deg}(i)}\sqrt{\widetilde{\deg}(j)}$ of the per-node form for each pair.

The operator $\tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$ in (7.4) connects the layer to the spectral perspective of Section 3.4. Setting the self-loops aside for a moment, the symmetric normalized Laplacian of Equation (3.13) is $\mathbf{L}_{\text{sym}} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$, so the propagation matrix without self-loops is exactly $\mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2} = \mathbf{I} - \mathbf{L}_{\text{sym}}$. A GCN layer therefore applies the operator $\mathbf{I} - \mathbf{L}_{\text{sym}}$ to the node representations before the learnable transformation. This is the sense in which the layer is *spectral*: it acts through a fixed function of the graph Laplacian, the operator whose spectrum Section 3.4 showed to be tied to the connectivity of the graph. The present work uses only the resulting operator $\mathbf{I} - \mathbf{L}_{\text{sym}}$ and does not develop the broader spectral construction from which it derives. The renormalization trick (7.3) addresses a difficulty of this operator: applying a propagation matrix of this form repeatedly, as a deep stack of layers does, can amplify or attenuate the representations to the point of numerical instability. Adding self-loops, which replaces $\mathbf{A}$ by $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ and the degree matrix accordingly, stabilizes the repeated application and is the form used in practice [2].

The spectral reading also explains the effect a GCN layer has on the representations. The quadratic form $\mathbf{f}^\top \mathbf{L}\mathbf{f}$ of Equation (3.11) measures how much an assignment of values to nodes varies across edges. The operator $\mathbf{I} - \mathbf{L}_{\text{sym}}$ attenuates the components of a signal associated with the large eigenvalues of the Laplacian — the components that vary most across edges — and preserves those associated with the small eigenvalues. Applying it therefore brings the representation of each node toward the average of its neighbors, reducing the variation across edges and making adjacent nodes more similar with every layer. This is *smoothing*: a GCN layer smooths the node representations over the edges of the graph.

Smoothing is the operative bias of GCN, and it is the right bias exactly when the graph is homophilous. Section 3.3 described a graph as homophilous when adjacent nodes tend to share attributes. On such a graph, making adjacent nodes similar aligns the representations with the labels, which is why GCN is effective for node classification despite the simplicity of its aggregation. The same mechanism becomes a liability when the assumption fails — when adjacent nodes differ systematically, or when so many layers are stacked that all representations are smoothed toward a common value — a limitation taken up in Chapter **??**.

[2] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the 5th International Conference on Learning Representations*, 2017. https://arxiv.org/abs/1609.02907

7.2 *Edge-Level Learning*

An edge-level task makes predictions about pairs of nodes rather than nodes individually. The representative task, introduced in Chapter 2, is link prediction: given the edges observed in a graph, predict which

further edges are likely to exist. A common approach to this setting is provided by the encoder–decoder framework, and requires two elements absent from the node-level case: a way to score a pair of nodes from their embeddings, and a way to construct a supervised training signal from a graph whose only data is the presence or absence of edges.

7.2.1 *The Encoder–Decoder Framework*

The framework divides a link-prediction model into two stages (Figure 7.1). The *encoder* is a message-passing network of the kind built in Chapter 6: it maps the graph to node embeddings,

$$\mathbf{Z} = \mathrm{ENC}(\mathbf{X}, \mathbf{A}), \tag{7.5}$$

where $\mathbf{Z} \in \mathbb{R}^{n \times d'}$ collects the final embeddings, its row i being the embedding $\mathbf{z}_i = \mathbf{h}_i^{(K)}$ of node i. The encoder carries all the graph structure into the embeddings; what remains is to turn a pair of embeddings into a prediction about the edge between them. The *decoder* performs this second step. It is a function that maps a pair of embeddings to a scalar score expressing how plausible an edge between the two nodes is. A suitable choice is the *inner product decoder* [3], which scores a pair by the inner product of their embeddings passed through a function that maps it to a probability,

$$\hat{A}_{ij} = \sigma\left(\mathbf{z}_i^\top \mathbf{z}_j\right), \tag{7.6}$$

where σ is the logistic function, mapping the real-valued score to a value in $(0, 1)$ read as the predicted probability of an edge between i and j. The inner product is large when the two embeddings point in a similar direction, so the decoder assigns high probability to pairs whose embeddings the encoder has placed close together. The decoder requires no parameters of its own.

To train it toward this arrangement, a supervised signal is needed, and the structure of the graph itself supplies it. The edges actually present in the graph are the *positive examples*: pairs that should receive a high score. Pairs of nodes with no observed edge between them are the *negative examples*: pairs that should receive a low score. A link-prediction model is trained to raise the score on positive pairs and lower it on negative pairs, exactly as a binary classifier is trained to separate two classes.

Two practical points govern how these examples are formed. First, the positive edges must be split, as any supervised problem splits its labels: a portion of the observed edges is held out from the graph given to the encoder and reserved for evaluation, so that the model is tested on its ability to recover edges it was not shown. The edges used for

[3] Thomas N. Kipf and Max Welling. Variational graph auto-encoders. In *NeurIPS Workshop on Bayesian Deep Learning*, 2016. https://arxiv.org/abs/1611.07308

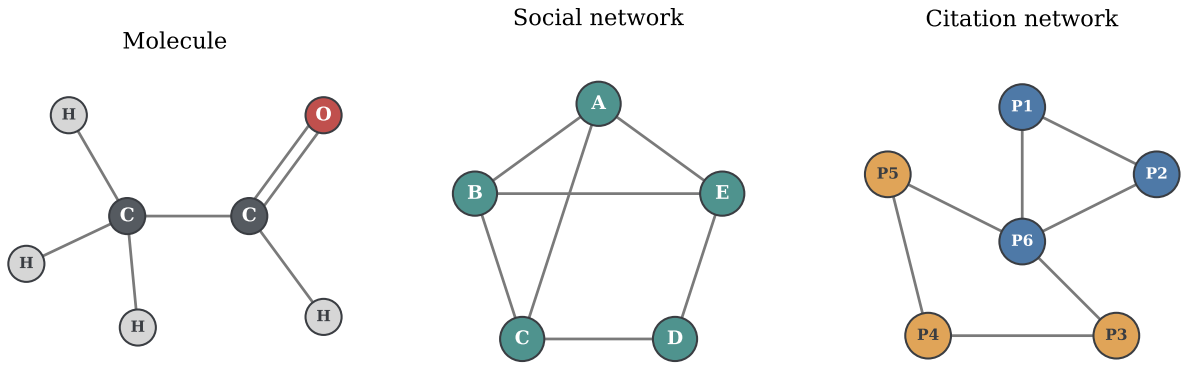

Figure 7.1: The encoder–decoder framework for link prediction. The encoder, a message-passing network, maps the graph $(\mathbf{X}, \mathbf{A})$ to node embeddings $\mathbf{Z}$, one vector $\mathbf{z}_i$ per node. The decoder takes a pair of embeddings $\mathbf{z}_i, \mathbf{z}_j$ and returns a scalar score $\hat{A}_{ij} = \sigma(\mathbf{z}_i^\top \mathbf{z}_j)$, the predicted probability of an edge between i and j. A high score is assigned where the encoder has placed the two embeddings close together.

evaluation are removed from $\mathbf{A}$ during training, not merely from the loss, since an edge left in the adjacency matrix would let the encoder see the very connection it is meant to predict. Second, the negative examples must be chosen, because they are far more numerous than the positive ones: a graph has at most $\binom{n}{2} = n(n-1)/2$ possible edges and typically far fewer actual ones, so the non-edges vastly outnumber the edges. Using all of them would unbalance the training signal. The standard remedy is *negative sampling*: at each step a subset of negative examples is drawn at random, in number comparable to the positive edges, so that the two classes contribute comparably to the training signal.

7.2.2 *The Graph Autoencoder*

The *Graph Autoencoder (GAE)* is the instance of the encoder–decoder framework that pairs a GCN encoder with the inner product decoder [4]. Its name comes from the autoencoder pattern: a model that encodes its input into a latent representation and then reconstructs the input from that representation. Here the input reconstructed is the adjacency matrix. The encoder is a GCN, producing the embedding matrix $\mathbf{Z}$ as in (7.5); the decoder applies the inner product (7.6) to every pair of embeddings at once, reconstructing the full adjacency matrix as

$$\hat{\mathbf{A}} = \sigma\left(\mathbf{Z}\mathbf{Z}^\top\right), \tag{7.7}$$

where σ is applied elementwise and the entry $\hat{A}_{ij}$ is the predicted probability of an edge between i and j.

Training matches the reconstructed adjacency against the observed one on the positive and sampled negative pairs of Section 7.2.1: the GCN encoder is fitted so that the embeddings it produces, when combined by the inner product, reproduce the edges of the graph. The

[4] Thomas N. Kipf and Max Welling. Variational graph auto-encoders. In *NeurIPS Workshop on Bayesian Deep Learning*, 2016. https://arxiv.org/abs/1611.07308

arrangement the encoder learns is the one the decoder discriminates — embeddings of connected nodes made similar, those of unconnected nodes made dissimilar — which is the arrangement a GCN produces by construction, since its smoothing brings the embeddings of adjacent nodes together.

7.3 Graph-Level Learning

A graph-level task assigns a single prediction to an entire graph. The representative task, introduced in Chapter 2, is graph classification: assigning a label to a whole graph, as in predicting a property of a molecule. This setting requires an element not needed in the previous levels — a way to reduce the per-node embeddings, which vary in number from graph to graph, to a single fixed-size vector representing the graph — and it places a demand on the message-passing layer that the previous tasks did not, since graph classification requires the layer to preserve the structural information that distinguishes one graph from another.

7.3.1 Pooling and Readout

A message-passing network produces one embedding per node. A graph-level prediction requires one vector for the whole graph, so the per-node embeddings must be combined into a single graph embedding. This reduction is the *readout* operation named in Section 6.2.4, also called *global pooling*: a function that maps the collection of node embeddings to one vector,

$$\mathbf{h}_{\mathcal{G}} = \text{READOUT}\Big(\{\, \mathbf{h}_i^{(K)} : i \in V \,\}\Big), \tag{7.8}$$

where $\mathbf{h}_{\mathcal{G}}$ is the graph embedding, on which a classifier of the form (7.1) is then applied to predict the graph's label.

The readout is subject to one essential requirement, fixed in Section 5.4: a graph-level prediction must be unchanged by a relabeling of the nodes, so the readout must be invariant to the order of its inputs. It takes a collection of node embeddings and must return the same vector whatever order they are listed in. This is the permutation invariance of Equation (5.4), and it restricts the readout to the order-insensitive operations of Section 6.2.2: the sum, the mean, and the maximum, taken coordinatewise over the node embeddings. The same property that made these operations suitable for aggregating a neighborhood makes them suitable for aggregating a whole graph; the readout is neighborhood aggregation applied once more, over all nodes at once.

The choice among sum, mean, and maximum is not neutral for a graph-level task, because the readout is the last point at which structural information can be preserved or lost, and the three operations differ in what they retain. The mean and the maximum can discard information that distinguishes graphs: the mean of a set is unchanged when the set is duplicated, and the maximum ignores all but the extreme value in each coordinate, so two graphs differing in the multiplicity or the distribution of their node embeddings can be mapped to the same vector. The sum retains more, because it grows with the number of nodes and reflects the full collection rather than its average or its extremes. This difference is decisive for the architecture that follows, whose design begins from the demand that no distinguishing information be discarded.

7.3.2 *The Graph Isomorphism Network*

The *Graph Isomorphism Network (GIN)* is the instance of the message-passing framework designed to retain as much distinguishing information as the framework allows [5]. Its motivation is the expressive-power question of Section 6.5: a graph-level model is only as good as its ability to tell non-isomorphic graphs apart, and Section 6.5 established that no message-passing network can exceed the discriminative power of the 1-WL test. GIN is the architecture that reaches this ceiling, and the requirement that fixes its design is the preservation of neighborhood information through the aggregation.

[5] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *Proceedings of the 7th International Conference on Learning Representations*, 2019. https://arxiv.org/abs/1810.00826

The requirement can be stated precisely with one notion not yet needed. A neighborhood, taken as the collection of its neighbors' representations, may contain the same representation more than once, since distinct neighbors may carry identical vectors. The right object is therefore not a set, which records only which elements are present, but a *multiset*, a collection in which elements may repeat and the number of repetitions is part of the object. For the aggregation to preserve all the information in a neighborhood, it must be *injective* over multisets: it must map distinct neighborhood multisets to distinct vectors, never collapsing two different neighborhoods to the same aggregate. The mean and the maximum fail this requirement, for the reason noted for the readout in Section 7.3.1: the mean cannot distinguish a neighborhood from its duplicate, and the maximum ignores all but the extreme value. The sum, by contrast, can be made injective over multisets, because it reflects every element and its multiplicity.

GIN builds its layer on the sum accordingly. The aggregation is the sum over the neighborhood; the update combines the node's own representation with this sum and passes the result through a function

expressive enough to preserve the distinction, giving

$$\mathbf{h}_i^{(k)} = \mathrm{MLP}^{(k)}\left(\left(1+\epsilon^{(k)}\right)\mathbf{h}_i^{(k-1)} + \sum_{j\in\mathcal{N}(i)} \mathbf{h}_j^{(k-1)}\right), \tag{7.9}$$

where $\mathrm{MLP}^{(k)}$ is the update function of the layer, and $\epsilon^{(k)}$ is a scalar — fixed or learnable — that weights the node's own representation against the summed neighborhood. The MLP in the update can approximate an injective function of its input, so the composition of the sum and the update preserves the distinction between different closed neighborhoods. The term $(1+\epsilon^{(k)})$ keeps the node's own representation distinguishable from the aggregate of its neighbors, rather than letting the two merge as they do in the closed-neighborhood sum of GCN. Together these choices make one GIN layer as discriminating as one round of the 1-WL refinement of Section 6.5, which is the most a message-passing layer can be.

The contrast with GCN is exact and is the point of the construction. GCN smooths, deliberately making adjacent nodes similar, which serves node classification on homophilous graphs but discards the neighborhood detail that distinguishes graphs. GIN refuses to smooth, preserving that detail instead, because its task is not to make neighbors similar but to tell structures apart.

This placement is the thread running through the three architectures of this chapter. All three are the same message-passing layer of Chapter 6. The design space, read as a space of such decisions rather than as a list of separate models, is what reveals the three architectures as instances of a single construction rather than three unrelated networks.

Bibliography

Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Caglar Gulcehre, Francis Song, Andrew Ballard, Justin Gilmer, George Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matt Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. Relational inductive biases, deep learning, and graph networks. arXiv:1806.01261 [cs.LG], 2018. URL https://arxiv.org/abs/1806.01261.

Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. ISBN 978-3-031-45467-7.

Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, 1997. ISBN 978-0-8218-0315-8.

Reinhard Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, 5 edition, 2017. ISBN 978-3-662-53622-3.

Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1263–1272, 2017. https://arxiv.org/abs/1704.01212.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. ISBN 978-0-262-03561-3.

William L. Hamilton. *Graph Representation Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool, 2020. ISBN 978-1-68173-965-6.

John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino

Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596 (7873):583–589, 2021. DOI: 10.1038/s41586-021-03819-2.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015. https://arxiv.org/abs/1412.6980.

Thomas N. Kipf and Max Welling. Variational graph auto-encoders. In *NeurIPS Workshop on Bayesian Deep Learning*, 2016. https://arxiv.org/abs/1611.07308.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the 5th International Conference on Learning Representations*, 2017. https://arxiv.org/abs/1609.02907.

Miller McPherson, Lynn Smith-Lovin, and James M. Cook. Birds of a feather: Homophily in social networks. *Annual Review of Sociology*, 27 (1):415–444, 2001. DOI: 10.1146/annurev.soc.27.1.415.

Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and Leman go neural: Higher-order graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 4602–4609, 2019. DOI: 10.1609/aaai.v33i01.33014602.

Christopher Morris, Yaron Lipman, Haggai Maron, Bastian Rieck, Nils M. Kriege, Martin Grohe, Matthias Fey, and Karsten Borgwardt. Weisfeiler and Leman go machine learning: The story so far. *Journal of Machine Learning Research*, 24(333):1–59, 2023. http://jmlr.org/papers/v24/22-0240.html.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *Proceedings of the 7th International Conference on Learning Representations*, 2019. https://arxiv.org/abs/1810.00826.

Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 974–983, 2018. DOI: 10.1145/3219819.3219890.

Marinka Zitnik, Monica Agrawal, and Jure Leskovec. Modeling polypharmacy side effects with graph convolutional networks. *Bioinformatics*, 34(13):i457–i466, 2018. DOI: 10.1093/bioinformatics/bty294.